

SIH 2026 · PS 26122 — Team Build Guide

Intelligent Data Capture & Schedule-Linking Layer

Oil India Limited · Software / Smart Automation

Five build days · Day 1 = Sunday 23 August 2026 · Day 5 = Thursday 27 August 2026

This is a practice run. The goal is that every member learns their part of the system and the team finds out what breaks — before the real build. Nothing here is judged.

Organised by person: read Part 0, then your own part. You do not need to read anyone else's.

How This Guide Is Organised

The blueprint is a contract for the whole team. This guide is the same contract, cut into sections addressed to individuals.

		page	length
PART 0	SHARED FOUNDATION everyone reads all of it Problem, product, architecture, scope, roles, the five-day shape, and the topics & tools every member must be familiar with	3	15 pp
PART 1	ARIN backend + database schema, dependency engine, correctness authority, presenter	18	21 pp
PART 2	SOUMYAK AI pipeline + frontend matcher, offline queue, presenter	39	11 pp
PART 3	AKSHAYA frontend + UI/UX design lead, independent verifier	50	7 pp
PART 4	KAUSHAL backend support under Arin learning track	57	6 pp
PART 5	TARUN lead, scope, domain, deck	63	8 pp
PART 6	KENNETH data, tests, evidence, deck	71	7 pp
PART 7	SHARED REFERENCE everyone reads this before Day 5 Test matrix, quality gates, completion definition, walkthrough beats, Q&A	78	9 pp

Part 1 is the longest because Arin owns every schema in the system. Nobody else needs to read those tables end to end.

Reading rule. Read Part 0 completely. Read your own part completely. Skim the parts of the two people you hand work to. Read Part 7 before Day 4 ends.

Because this is a practice run, every day also lists *what you should have learned by the end of it*. Missing a target is not a failure — not knowing **why** you missed it is. Write the reason down.

Precedence rule. If this guide and the running code disagree, stop building and resolve the mismatch. If a feature is not in the Tier 1 scope contract and does not protect the end-to-end pipeline, Tarun decides whether it is cut.

The Five-Day Finish Line

Everything in this guide exists to make this one chain work without anybody editing the database by hand.



Acceptance rule. If the team cannot demonstrate this chain without manually editing the database, the five-day MVP is not complete.

PART 0 — SHARED FOUNDATION

Everyone reads this part in full

0.1 The Problem in One Sentence

Oil India can have a detailed digital project plan, but real construction progress arrives as messy human language, DPRs, spreadsheets, photos, calls and contractor messages. The system captures those updates, links them to the correct planned L5/L6 activities, asks a human to verify the link, and then updates Plan vs Actual progress and downstream delay impact.

0.2 What We Are Building

An **AI-assisted, human-approved reconciliation layer** between field execution and project-control systems.

Not another generic project-management dashboard. The central technical product is the **schedule-linking and granularity-reconciliation engine**:

1. understand a messy field report;
 2. extract one or more structured progress events;
 3. retrieve and rank possible L5/L6 schedule activities;
 4. map many small field events to one larger planned activity where necessary;
 5. expose confidence, alternatives, source evidence and reasoning;
 6. require Confirm, Correct or Reject;
 7. write only approved facts to the actuals record;
 8. run deterministic scheduling maths and update the management view;
 9. preserve every raw report, proposal, decision and correction for audit and learning.
-

0.3 The Product Promise

Turn delayed, fragmented, manually reconciled field reporting into timely, attributable, schedule-linked actual progress — without letting AI silently rewrite the plan.

0.4 What "Real-Time" Honestly Means

The product is as current as its input. It cannot observe work that nobody reports.

"Real-time" means	"Real-time" does not mean
Immediate capture when a worker submits an update	Continuous autonomous site surveillance
Offline-safe storage when internet is unavailable	Detecting work nobody reported
Automatic synchronisation when connectivity returns	Instant AI processing while offline
Prompt AI processing after synchronisation	AI writing schedule facts on its own
Immediate review-queue creation	
Immediate schedule and dashboard refresh after human confirmation	

0.5 Plan and Reality Speak Different Languages

The planned world

```

PROJECT: Well-Site / Pipeline EPC
├── L2: Civil
│   ├── L3: Area A
│   │   └── L5: Foundations
│   │       ├── L6 CIV-A-101 Excavation
│   │       ├── L6 CIV-A-102 Rebar and formwork
│   │       └── L6 CIV-A-103 Foundation concreting
│   └── L2: Mechanical
│       └── L6 MEC-A-201 Install Pump P-101
├── L2: Piping
│   └── L6 PIP-B-301 Weld 24-inch separator line
└── L2: Electrical & Instrumentation
    └── L6 ELE-A-401 Terminate Pump P-101 feeder
  
```

Each activity has a controlled identity, dates, duration, discipline, zone, predecessors and responsibility.

The executed world

```

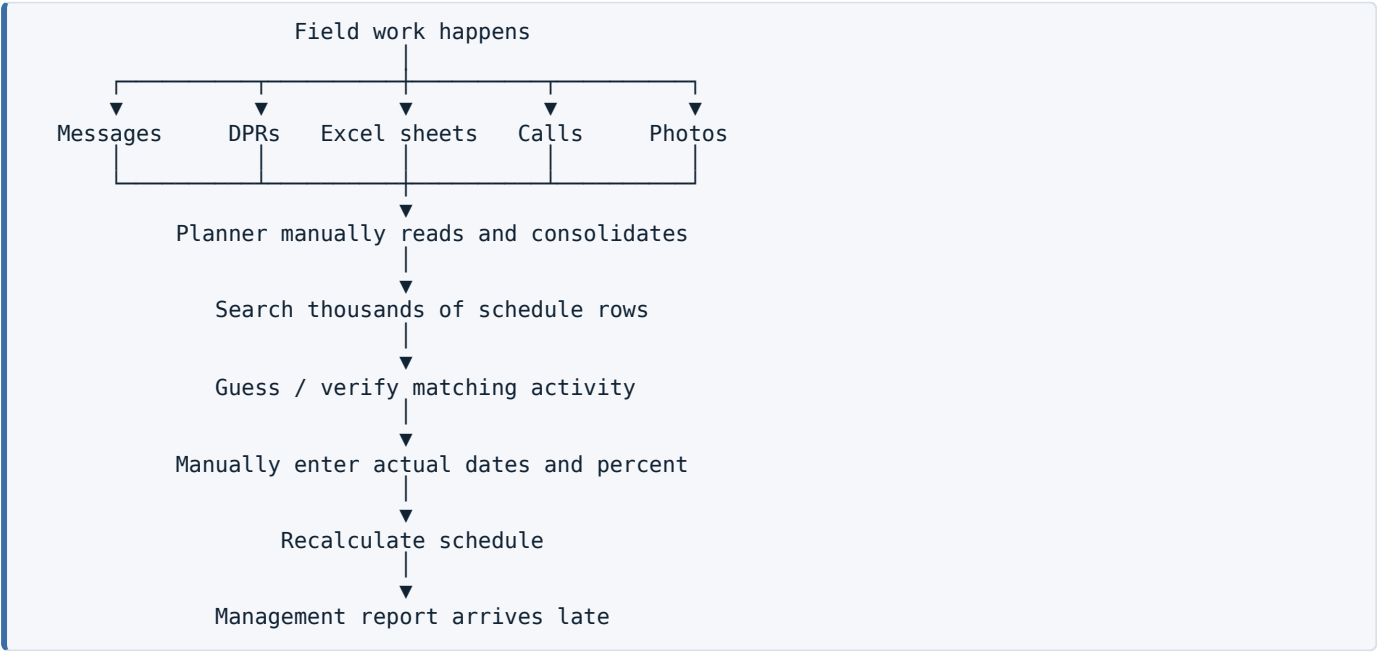
"Zone 3 foundation concreting khotom, curing shuru kal se."
"P201 alignment done. Cable termination pending."
"Separator side main line welding finished today."
"Material not received; trench team stopped."
"About 60% of the north-side pipe supports erected."
  
```

The same physical fact can arrive as free text, a chat message, a Daily Progress Report, a contractor Excel sheet, voice transcription, a photograph and caption, site diary notes, or a low-bandwidth SMS.

Where the gap appears

Planned data	Field data	Reconciliation difficulty
Controlled task ID	No ID or wrong shorthand	Entity resolution
Standard activity name	Slang, abbreviations, code-mixing	Language normalisation
One L5/L6 activity	Several smaller site actions	Granularity reconciliation
Planned dates	"today," "yesterday," "since morning"	Temporal extraction
Status codes	"done," "started," "held," "70%"	Event / status extraction
Formal discipline and location	Nicknames and asset tags	Context resolution
Dependency graph	Narrative delay reason	Delay propagation
Controlled change process	Many reporters and sources	Trust, governance, audit

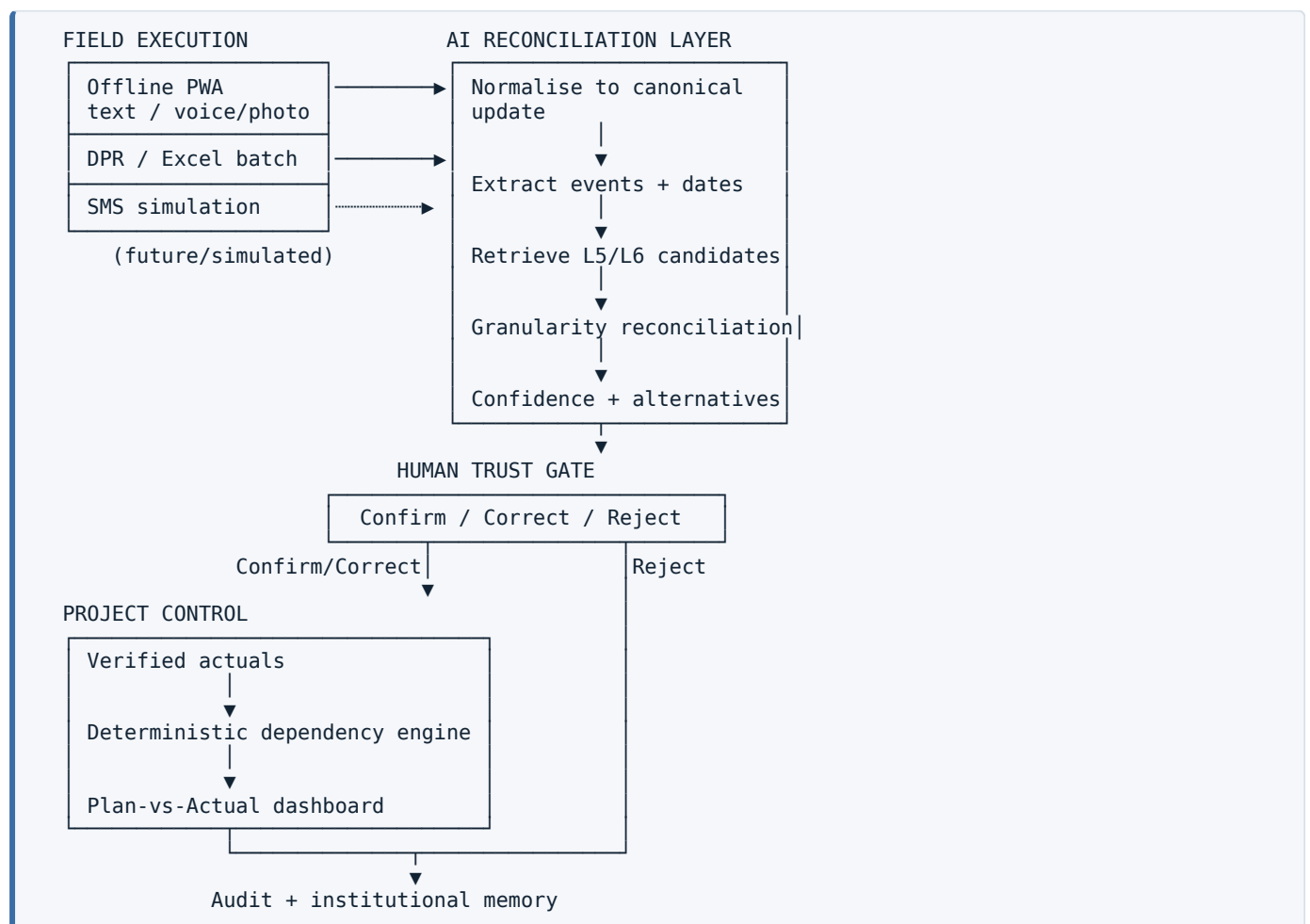
Current-state workflow



The resulting harm

Impacted group	Current pain	Consequence
Field worker / supervisor	Reporting tools too complex or connectivity weak	Updates delayed, shortened or lost
Site engineer	Repeats the same status in several formats	Time lost, inconsistent records
Planning engineer	Manually links informal reports to formal IDs	Reconciliation backlog, wrong mappings
Project manager	Sees stale consolidated data	Slow corrective action
Discipline lead	Teams describe the same work differently	Duplicate or contradictory progress
Contractor	Progress evidence not attributable	Disputes over completion and dates
Senior management	Late Plan-vs-Actual reporting	Poor portfolio decisions, surprise delays
Auditor / governance	Cannot trace who changed what and why	Weak accountability
Future project team	Lessons stay in documents and people's memories	Repeated estimation and execution mistakes

Target-state workflow



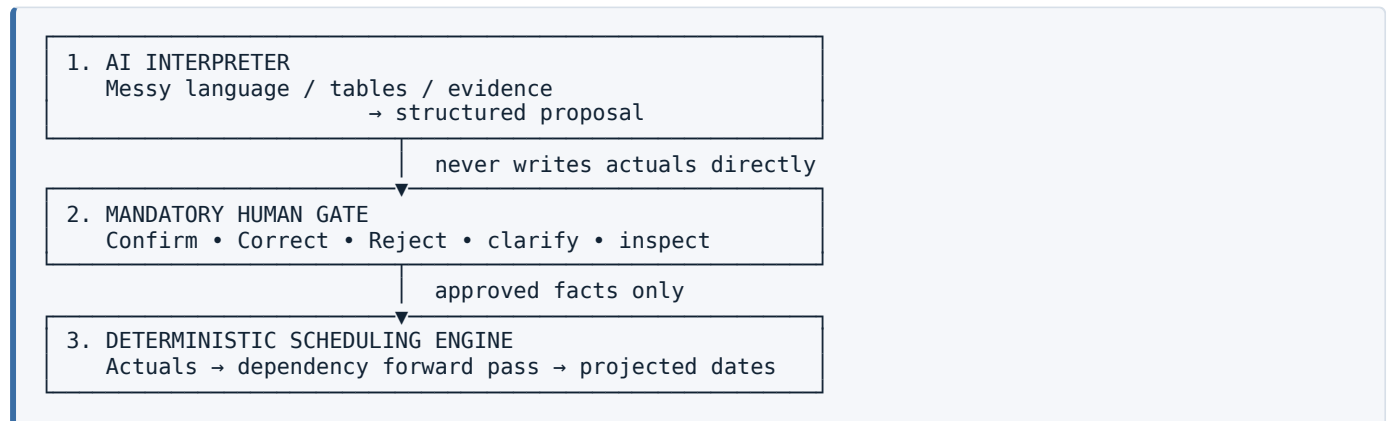
0.6 Product Thesis

Working category: Field-Update-to-Gantt Automation / Planning-to-Execution Bridge

Core capability: AI-assisted reconciliation of actual field events against a formal activity graph

Primary interfaces: Offline-first Field Mode and online Control Room Mode

Three-part safety architecture



Ten non-negotiable principles

1. **Data first.** Decide schemas and state transitions before generating screens.
2. **AI proposes; authorised humans decide.** No LLM output directly changes schedule actuals.
3. **Scheduling maths is not AI.** Dependency propagation is deterministic, testable code.
4. **Raw evidence is immutable.** Corrections append new records; they never erase history.
5. **Offline means safe capture, not offline cloud AI.** Processing can wait for connectivity.
6. **Uncertainty is a product state.** "Needs clarification" beats a confident wrong match.
7. **Baseline is sacred.** Planned dates never change; actual and projected are separate fields.
8. **One canonical event format.** Every channel enters the same downstream pipeline.
9. **Idempotency everywhere.** A retry or double-click must not create duplicate actuals.
10. **Honest demo labels.** Synthetic data, simulated transports and unsupported integrations are named clearly.

The process constraint

All production code is AI-generated; humans direct, read, verify and test it. A wrong result is corrected through a smaller, clearer generation request and re-verification — never by hand-patching. **Arin remains the authority on whether generated code is correct.**

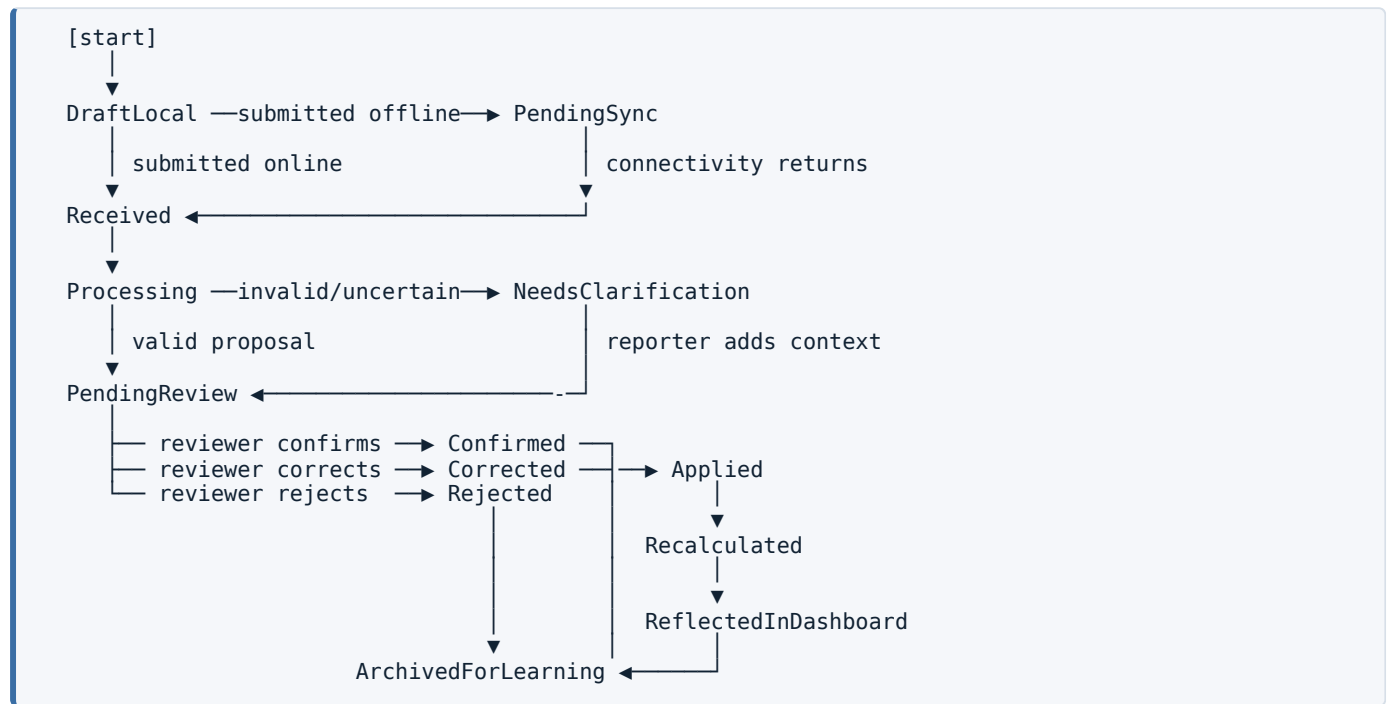
0.7 System Architecture — Follow the Data



Runtime boundaries

Boundary	Runs where	Holds what	Why
PWA client	Browser / installed PWA	app shell cache, reference subset, draft updates, sync queue	field capture survives weak connectivity
Supabase	Cloud backend	schedule, raw updates, proposals, actuals, audit, attachments	shared source of truth
Edge Functions	Server side	import validation, AI call, output validation, approval transaction, recalculation	secrets and trusted logic stay off client
LLM provider	External API for MVP	compact schedule context and update text	language interpretation only
Analytics UI	Browser	read-only aggregates and visualisations	immediate project-control visibility

Data lifecycle and trust level



Trust rule. `field_updates` and `extracted_events` are *claims*. `proposals` are *AI suggestions*. Only `activity_actuals` contains *approved schedule facts*.

0.8 MVP Scope Contract

Tier 1 — must work by the end of Day 5

#	Capability	Minimum working proof
1	Schedule CSV/XLSX import	Upload a 40-task synthetic EPC schedule; validate, preview, commit
2	WBS + dependency representation	L5/L6 activities; graph with 6+ levels and parallel branches
3	Free-text field capture	Submit informal and code-mixed progress text
4	DPR / Excel ingestion	Multiple rows or report lines become separate canonical updates
5	Actual event extraction	START, FINISH, PROGRESS, DELAY, BLOCKED, RESUMED + date, percent, asset, zone, discipline
6	L5/L6 matching	Rank correct activity plus 2–3 alternatives using constrained candidates
7	Confidence and clarification	High / Medium / Low plus explicit <code>needs_clarification</code>
8	Granularity reconciliation	Map multiple small field events to a larger parent with configurable weights
9	Human trust gate	Confirm, Correct, Reject; no bypass
10	Actuals write	Approved actual start/finish/progress stored transactionally and idempotently
11	Audit trail	Raw input, proposal, decision, actor, time, evidence, before/after values
12	Dependency / delay propagation	FS forward pass updates projected dates and project finish
13	Plan-vs-Actual view	Baseline vs actual/projected Gantt, variance, at-risk list, KPIs
14	Offline-first PWA capture	Shell loads offline; update persists in IndexedDB and syncs later
15	Historical dataset	Approved/rejected mappings and execution facts queryable
16	Reset and fallback	One-click demo reset plus recorded or cached fallback artifacts

Tier 2 — only after Tier 1 is stable

Photo upload with AI-generated **description** as human-reviewable evidence · voice recording/transcription · multilingual and Assamese-English / Hindi-English expansion · richer progress weighting and milestone configuration UI · historical analytics charts and repeated-delay insights · simulated inbound SMS endpoint and button-phone demo card · export of verified actuals to CSV · second demo scenario.

Tier 3 — roadmap, not a five-day claim

Real SMS / IVR / USSD telecom integration · direct Primavera P6 or MS Project API writeback · real WhatsApp Business integration · native Android/iOS app · autonomous visual verification of construction completion · on-premise models and enterprise deployment · full RBAC, SSO, multi-project / multi-tenant · complete CPM with SS/FF/SF, leads, lags, resource levelling and calendars · BIM / digital-twin integration · large-scale vector retrieval over tens of thousands of activities.

Explicitly out of scope

Not being built	What is built instead
Direct live Primavera integration	Validated CSV/XLSX import compatible with common schedule exports
Native mobile application	Installable responsive PWA
Full offline AI	Offline capture and deferred processing after sync
Telecom production integration	Optional simulated SMS path; IVR/USSD architecture only
AI-authored schedule changes	Human-approved actual updates only
AI completion verification from photos	Neutral description and evidence display
Every dependency type	Finish-to-Start with lag; schema stays extensible
Production security certification	Sensible secrets, auth, audit and deployment notes for a prototype

0.9 The Team

Person	Level	Role	Authority
Arin	Strongest coder (1st)	Backend and database. Schema, dependency engine, Edge Function, review of all generated code. Presenter.	Technical correctness authority
Soumyak	2nd strongest coder; experienced vibe coder; strong speaker	AI pipeline (extraction + L5/L6 matcher) and frontend with Akshaya. Presenter.	AI pipeline and generation authority
Akshaya	Intermediate Java/C/C++; Canva expert	Frontend and UI/UX with Soumyak. Design lead. Independent verifier of the scheduling maths.	UI authority and independent maths check
Kaushal	3rd; learning. Uses Claude Code. Built a credit-card recommender	Backend support under Arin — small, self-contained, reviewed tasks. Learning track.	None held alone; works under Arin's review
Tarun	PPT; some coding; strong judgment	Team lead, scope controller, domain-knowledge owner. Deck with Kenneth.	Scope, taxonomy, evidence-policy and final cut authority
Kenneth	PPT; research, documentation, structured support	Data, test set, fixtures, test execution. Deck with Tarun.	Evidence, dataset and testing authority

How the work splits

BACKEND / DATABASE	Arin (lead) + Kaushal (support)
AI PIPELINE	Soumyak
FRONTEND / UI / UX	Soumyak + Akshaya
INDEPENDENT MATHS CHECK	Akshaya
DATA, FIXTURES, TESTS	Kenneth
SCOPE, DOMAIN, DECK	Tarun + Kenneth
WALKTHROUGH PRESENTERS	Soumyak + Arin

Pairing 1 — Arin and Kaushal. Arin defines a small, self-contained task with a clear input and output; Kaushal builds it in Claude Code; Arin reviews every line. Never more than one task at a time, never anything on the critical path.

Pairing 2 — Soumyak and Akshaya. She decides what a screen should look like and be; he builds the harder wiring. Neither waits on the other for a whole screen.

Independent check — Akshaya on Arin. She hand-calculates the schedule cascades before seeing the engine's answer, so the maths is never self-reviewed.

Presenters — Soumyak and Arin. Soumyak covers the narrative and the AI pipeline; Arin covers architecture, the database and the scheduling engine.

Responsibility matrix

A accountable · **R** responsible · **C** consulted · **V** independent verifier · **I** informed

Workstream	Arin	Soumyak	Akshaya	Kaushal	Tarun	Kenneth
Scope and acceptance contract	C	I	C	I	A/R	I
Data schema and migration	A/R	C	V	R	C	I
Synthetic schedule	C	I	V	C	C	A/R
Schedule import (backend)	A/R	C	I	R	I	C
Schedule import (UI)	C	R	A/R	I	I	C
AI extraction and matching	A/V	A/R	I	I	C	C
Edge Function implementation	A/R	C	I	R	I	I
Dependency engine	A/R	C	V	C	C	I
Granularity engine	A/R	R	V	C	C	I
Frontend / review UI	C	R	A/R	I	C	I
Dashboard / Gantt	C	R	A/R	I	C	I
Offline capture / sync	A/C	A/R	C	I	I	I
Audit trail	A/R	C	C	R	C	I
Evidence policy	C	I	C	I	A/R	C
Test set and gold labels	C	R	I	R	C	A/R
Test execution	C	R	C	R	I	A/R
Demo integration / reset	A/R	R	C	C	A	C
Deck and slides	C	I	R	C	A/R	A/R
Walkthrough delivery	R	A/R	I	I	C	C
Final triage / cut decisions	C	C	C	I	A/R	I

Tool stack locked for the week

Layer	Chosen tool	Owner	Note
Database, auth, storage	Supabase (PostgreSQL)	Arin	The source of truth for everything
Backend logic	Supabase Edge Functions (Deno/TypeScript)	Arin; Kaushal on scoped tasks	Secrets and trusted logic stay off the client
Frontend + app generation	Lovable.dev	Soumyak + Akshaya	Bolt.new if Lovable blocks progress
Web app	React/TypeScript, generated	Soumyak; Arin reviews	Responsive installable PWA, not native
Code generation	Claude Code	Everyone who writes code	Kaushal uses this for his scoped tasks
LLM in the product	Claude API	Soumyak prompt; Arin validation	Low temperature, strict JSON, null rather than guess
Gantt rendering	frappe-gantt	Akshaya	Baseline overlay is custom work — see Part 3
Schedule authoring	Google Sheets → CSV/XLSX	Kenneth; schema by Arin	Synthetic but realistic OIL-style EPC data
Spreadsheet parsing	SheetJS (<code>xlsx</code>)	Soumyak; Arin reviews	Preview before database commit
Offline storage	Service worker + IndexedDB	Soumyak; Arin reviews sync contract	Manual retry fallback
Evaluation scoring	Google Sheets scorecard, or a small script	Kenneth records; Soumyak runs	Version every scored prompt
Deck and visuals	Canva + Gamma	Tarun + Kenneth	Claims use measured numbers only

The build rule. All production code is AI-generated; humans direct, read, verify and test it. A wrong result is corrected by re-scoping the generation request smaller — never by hand-patching. Arin is the authority on whether generated code is correct.

0.10 Topics and Tools Every Member Must Be Familiar With

No one on this team writes code by hand. That does **not** remove the need to understand what is being built. It changes what you need to understand: less syntax, more *concepts*, *contracts* and *failure modes*. You cannot direct an AI to build something you cannot describe, and you cannot tell whether it built the right thing if you do not know what right looks like.

Everyone — the shared baseline

Every member should be able to explain each of these in one or two sentences. This is a hard requirement, not a suggestion; on Day 5 anyone should be able to answer a question about any of them.

#	Topic	What "familiar" means
1	WBS and L5/L6 levels	Why L1–L4 are summary nodes and L5/L6 are the only levels a date can be written to
2	Finish-to-Start dependency and lag	If A finishes late, B starts late; lag is the gap between them
3	Baseline vs actual vs projected	Three separate date fields. Baseline never changes. That is what makes variance visible
4	What a Gantt bar encodes	Start, duration, progress fill, dependency arrow, today line
5	Percent complete vs earned progress	Reported percent is a claim; earned progress is the sum of approved milestone weights
6	The human gate	AI proposes, a human approves, only then does anything get written
7	Idempotency	Clicking Confirm twice, or a retry after a timeout, must not apply the change twice
8	Audit trail / append-only	Corrections add new rows; nothing is ever overwritten or deleted
9	Structured output / JSON schema	The model must return a fixed shape, and the server validates it before use
10	Confidence and abstention	"I don't know" is a valid, and safer, answer than a confident wrong one
11	Held-out test set	Why the test cases must be written <i>before</i> the prompt is tuned
12	Top-1 vs top-3 accuracy	Did we get it right first try, vs was the right answer in the shortlist
13	Real vs synthetic vs simulated	Which parts of our demo are which, and saying so out loud first
14	Offline-first capture	The app stores locally and syncs later; AI processing waits for connectivity
15	Why scheduling is not AI here	Language is interpreted by a model; dates are computed by deterministic code

Everyone — tools to have opened at least once

Tool	What you should be able to do
Supabase	Open the project, find the table editor, read a table, find the logs
Lovable.dev	Understand what a prompt does to the app, even if you never drive it
Claude Code	Ask for a small change, read the diff it proposes, accept or reject
Google Sheets → CSV/XLSX	Export cleanly, know why date formats break imports
Browser devtools	Open the console and the network tab; spot a failed request
Canva / Gamma	Build or edit a slide without asking someone else
The deployed app itself	Run the full end-to-end flow yourself, at least once, before Day 5

Per member — what you specifically must be fluent in

Member	Must be fluent in
Arin	Relational schema design · foreign keys and join tables · transactions and rollback · topological sort and cycle detection · forward-pass date arithmetic · idempotency keys and unique constraints · server-side validation · where API keys live · reading generated TypeScript critically
Soumyak	Prompt design for structured extraction · few-shot examples · candidate filtering and ranking · confidence calibration · JSON-mode enforcement · Lovable prompt shapes against an existing schema · service workers and IndexedDB · reading an error breakdown and turning it into a change
Akshaya	Design tokens and visual hierarchy · projector legibility · human-in-the-loop review UX · frappe-gantt task objects and options · date-to-pixel arithmetic for the baseline overlay · inclusive vs exclusive end dates · hand-calculating a schedule cascade · reading code well enough to spot an off-by-one
Kaushal	Reading a schema and knowing which table a field belongs to · scoping one task small enough to be reviewable · using Claude Code to change one file at a time · reading a diff before accepting it · SQL SELECT well enough to check your own work · what a REST endpoint is and what it returns
Tarun	EPC activity vocabulary and site phrasing · WBS meaning · what evidence is sufficient per discipline · audit and attributable change · scope control and cut discipline · structuring a technical deck
Kenneth	WBS and dependency basics · what makes a synthetic schedule plausible · gold-label discipline · the difference between accuracy and "it worked when I tried it" · reproducible test execution · tracing every claim in the deck to a source

0.11 The Five Days

DAY 1	Sun 23 Aug	FOUNDATION Schema, synthetic schedule, import, app shell
DAY 2	Mon 24 Aug	INTELLIGENCE Canonical ingestion, extraction, L5/L6 matcher, measured accuracy
DAY 3	Tue 25 Aug	CORE ENGINES Granularity, deterministic dependency engine, offline queue
DAY 4	Wed 26 Aug	TRUST + PRODUCT Review gate, transactional actuals, audit, dashboard
DAY 5	Thu 27 Aug	INTEGRATION + RETRO Full end-to-end run, walkthrough, and a written retrospective – no new features

The gate at the end of each day

Day	The one thing that should be true
1	40 tasks imported through the UI , Gantt reads live from Supabase, zero cycles
2	≥60 held-out messages scored against a named prompt version
3	A 3-day mid-chain delay shifts downstream tasks exactly as hand-calculated
4	Text → proposal → Confirm → actual → cascade → dashboard → audit, with no DB editing
5	Two clean end-to-end runs after reset, and a written retro

This is a practice run, so a missed gate is information, not failure. But it is only information if you write down *why* it was missed. A gate missed with no recorded reason is a wasted day. If a day's gate fails, the next day starts as repair work — not as new feature work.

The Day 5 retrospective — the actual point of this week

At the end of Day 5, the team writes one page answering:

1. Which gate did we miss, and what was the real cause?
(not "we ran out of time" – what specifically took longer than expected, and why)
2. What did each person underestimate about their own part?
3. Which task turned out to be on the critical path that we did not expect to be?
4. What would we do differently on Day 1 of the real build?
5. Which tool fought us, and is it worth replacing?

Keep this page. It is worth more to the real build than the code you write this week.

PART 1 — ARIN

Backend and database · schema owner · dependency engine · code reviewer · presenter

1.0 Your Mandate

You own **the backend and the database**, and you are the **technical correctness authority**. Nothing ships that you have not judged correct. You are the strongest coder here, so your time goes on the parts where being wrong is expensive — the schema, the write path, and the scheduling engine.

You are not on the frontend. Soumyak and Akshaya own that entirely. If a screen is wrong, tell them what it should do; do not open Lovable.

What only you decide

- the database schema and every state transition;
- whether a generated function is correct enough to keep;
- the transaction boundary around approving a proposal;
- whether the dependency engine's output is trustworthy;
- where secrets live and what the client is allowed to see.

What you must not spend your days on

Frontend work, UI decisions, and prompt engineering. Those belong to Soumyak and Akshaya. Your time is the scarcest correctness resource on the team.

Working with Kaushal

Kaushal is third in coding ability and is here to learn. He works under you, on the backend, using Claude Code. The rule that makes this work rather than cost you time:

```
ONE task at a time
SELF-CONTAINED — one function or one endpoint, not a feature
CLEAR CONTRACT — you state the input, output and failure shape
OFF the critical path — if it is late, the demo still works
REVIEWED — you read every line before it merges
```

Good tasks for him: a validation helper, a single GET endpoint, a seed script, an audit-log writer, a CSV column normaliser. Bad tasks: the dependency engine, the approval transaction, anything touching idempotency.

Budget 20–30 minutes a day reviewing his work. If it starts costing more than that, shrink the tasks — do not take them back silently.

Your two irreplaceable outputs: the deterministic dependency engine (Day 3) and the atomic approval transaction (Day 4). If either is wrong, the walkthrough is a lie and the product does not exist.

You are also one of the two presenters. Soumyak takes the narrative and the AI pipeline; you take architecture, the database and the scheduling engine. See §1.12.

1.1 Identifier and Timestamp Rules

Decide these on Day 1 morning and never revisit them.

- Use UUIDs for database records.
- Preserve imported business IDs such as `PIP-L6-2387` as unique project-scoped strings.
- Every offline submission carries a client-generated UUID used as an **idempotency key**.
- Store timestamps in UTC; render in the project timezone (`Asia/Kolkata`).
- Preserve raw source text and raw imported row JSON unchanged.
- Decisions and audit entries are **append-only**; never overwrite history.
- Planned, actual and projected dates are **three separate fields**, never one.

1.2 Core Relational Schema

projects

Column	Type	Rules / meaning
<code>id</code>	uuid	primary key
<code>name</code>	text	required
<code>code</code>	text	unique
<code>timezone</code>	text	default <code>Asia/Kolkata</code>
<code>baseline_start</code>	date	required
<code>baseline_finish</code>	date	required
<code>status</code>	enum	draft / active / archived
<code>created_at</code>	timestamp _{tz}	server timestamp

wbs_nodes

Column	Type	Rules / meaning
id	uuid	primary key
project_id	uuid	foreign key
wbs_code	text	project-scoped unique code
name	text	required
level	integer	1-6
parent_id	uuid nullable	self-reference
discipline	text nullable	civil / mechanical / piping / electrical / instrumentation / commissioning
zone	text nullable	normalised location

activities

Column	Type	Rules / meaning
id	uuid	primary key
project_id	uuid	foreign key
wbs_node_id	uuid	usually L5/L6 parent
activity_code	text	imported ID; unique per project
name	text	formal schedule description
description	text nullable	expanded context
discipline	text	controlled value
zone	text	controlled value
asset_tag	text nullable	pump / line / equipment identifier
planned_start	date	immutable baseline
planned_finish	date	immutable baseline
duration_days	integer	positive
status	enum	not_started / in_progress / complete / blocked / at_risk
percent_complete	numeric(5,2)	0-100; approved value only
actual_start	date nullable	approved only
actual_finish	date nullable	approved only
projected_start	date nullable	engine output
projected_finish	date nullable	engine output
responsible_crew	text nullable	schedule context
search_text	text	normalised matching text

activity_dependencies

Column	Type	Rules / meaning
id	uuid	primary key
project_id	uuid	foreign key
predecessor_activity_id	uuid	required
successor_activity_id	uuid	required
dependency_type	enum	MVP accepts/stores FS; schema can hold SS/FF/SF
lag_days	integer	default 0
constraint	—	predecessor $\neq$ successor; unique edge

Day 1 decision that pays off all week: use this join table rather than a `predecessor_ids` column. Many-to-many predecessors become trivial on Day 3.

subtask_templates

Defines the small field events that contribute to one larger activity.

Column	Type	Rules / meaning
id	uuid	primary key
parent_activity_id	uuid	larger planned activity
code	text	e.g. POSITION , ALIGN , BOLT
name	text	human label
sequence_no	integer	normal order
weight_percent	numeric(5,2)	all active templates for a parent total 100
is_start_trigger	boolean	can set parent actual start after approval
is_finish_required	boolean	required before derived finish can be proposed
synonyms	jsonb	site-language variants

field_updates

Column	Type	Rules / meaning
id	uuid	server primary key
client_event_id	uuid	unique idempotency key
project_id	uuid	foreign key
source_type	enum	pwa_text / dpr / xlsx / csv / voice / photo / sms_simulated
source_filename	text nullable	batch provenance
source_row_number	integer nullable	batch traceability
raw_text	text	immutable source text
raw_payload	jsonb	original row / metadata
reporter_id	uuid nullable	user identity
reported_at	timestampz	stated / report time
received_at	timestampz	server time
device_created_at	timestampz	offline creation time
sync_status	enum	received / queued / processed / failed
processing_error	text nullable	visible, retryable failure

attachments

Column	Type	Rules / meaning
id	uuid	primary key
field_update_id	uuid	foreign key
storage_path	text	protected object path
mime_type	text	image / audio / document
sha256	text	deduplication / integrity
ai_description	text nullable	observable description only
metadata	jsonb	size, dimensions, duration

extracted_events

One update can contain multiple events: "*alignment complete; termination pending.*"

Column	Type	Rules / meaning
id	uuid	primary key
field_update_id	uuid	foreign key
event_index	integer	order within update
event_type	enum	START / FINISH / PROGRESS / DELAY / BLOCKED / RESUMED / NO_EVENT
activity_phrase	text	phrase describing the work
event_date	date nullable	resolved date
percent_value	numeric nullable	0-100
asset_tag	text nullable	extracted equipment / line tag
zone	text nullable	normalised; raw retained in JSON
discipline	text nullable	inferred discipline
delay_days	integer nullable	stated delay
delay_reason	text nullable	material / access / weather / etc.
evidence_refs	uuid[]	attachment links
extractor_version	text	prompt / model version
raw_extraction	jsonb	full validated model output

match_candidates

Column	Type	Rules / meaning
id	uuid	primary key
extracted_event_id	uuid	foreign key
activity_id	uuid	candidate activity
rank	integer	1, 2, 3...
semantic_score	numeric	0-1
asset_score	numeric	0-1
zone_score	numeric	0-1
discipline_score	numeric	0-1
chronology_score	numeric	0-1
state_score	numeric	0-1
final_score	numeric	calibrated 0-1
explanation	text	concise, reviewer-facing

proposals

Column	Type	Rules / meaning
id	uuid	primary key
extracted_event_id	uuid	foreign key
proposed_activity_id	uuid nullable	top match; nullable when uncertain
proposed_subtask_template_id	uuid nullable	granularity link
proposed_event_type	enum	same controlled event types
proposed_event_date	date nullable	extracted date
proposed_percent	numeric nullable	derived or reported
confidence	numeric	0-1
confidence_band	enum	high / medium / low
needs_clarification	boolean	required when unsafe to choose
status	enum	pending / confirmed / corrected / rejected / superseded
prompt_version	text	reproducibility
created_at	timestampz	server time

review_decisions

Column	Type	Rules / meaning
id	uuid	primary key
proposal_id	uuid	foreign key
decision	enum	CONFIRM / CORRECT / REJECT
selected_activity_id	uuid nullable	chosen activity after confirmation/correction
selected_subtask_template_id	uuid nullable	corrected granularity link
approved_event_type	enum nullable	final event type
approved_event_date	date nullable	final date
approved_percent	numeric nullable	final percent
reason	text nullable	mandatory for reject; encouraged for correction
decided_by	uuid	authorised reviewer
decided_at	timestamp tz	server timestamp

activity_actuals

Column	Type	Rules / meaning
id	uuid	primary key
activity_id	uuid	foreign key
review_decision_id	uuid	unique — prevents double-apply
event_type	enum	approved event
event_date	date nullable	approved actual date
percent_delta_or_value	numeric nullable	explicit semantics stored
source_field_update_id	uuid	provenance
applied_at	timestamp tz	server time

subtask_actuals

Column	Type	Rules / meaning
id	uuid	primary key
parent_activity_id	uuid	larger activity
subtask_template_id	uuid	small milestone
review_decision_id	uuid	approved source
status	enum	not_started / in_progress / complete / rejected
event_date	date	approved date
earned_weight	numeric	template weight when complete

delay_impacts

Column	Type	Rules / meaning
id	uuid	primary key
trigger_actual_id	uuid	event causing recalculation
affected_activity_id	uuid	downstream activity
old_projected_start	date	before value
new_projected_start	date	after value
old_projected_finish	date	before value
new_projected_finish	date	after value
variance_days	integer	impact
calculation_version	text	reproducibility

audit_log

Column	Type	Rules / meaning
id	uuid	primary key
project_id	uuid	foreign key
actor_id	uuid nullable	user / system identity
action	text	UPDATE_RECEIVED / PROPOSAL_CREATED / CONFIRMED / CORRECTED / REJECTED / ACTUAL_APPLIED / RECALCULATED
entity_type	text	table / domain object
entity_id	uuid	affected object
before_state	jsonb nullable	previous values
after_state	jsonb nullable	new values
source_ref	uuid nullable	update / decision link
created_at	timestampz	immutable server time

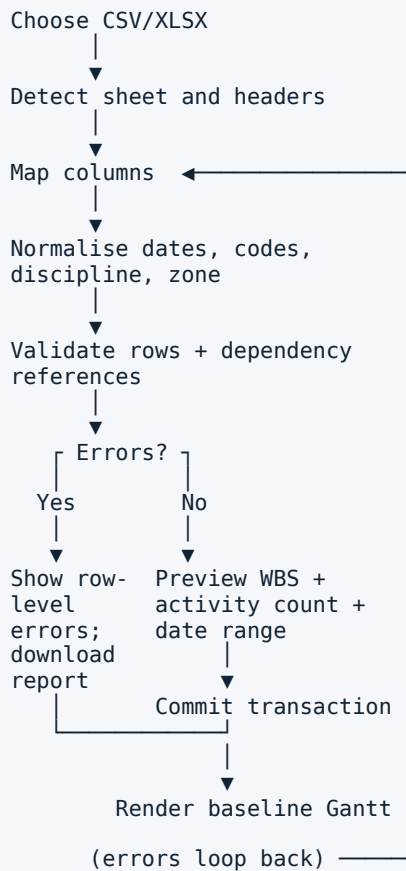
prompt_versions , evaluation_cases , evaluation_runs

These make measured accuracy reproducible — the difference between "82%" being a claim and being evidence.

Table	Essential columns
prompt_versions	id , name , version , model , temperature , schema_version , prompt_hash , created_at
evaluation_cases	id , message , gold_activity_id , gold_event_type , gold_date , difficulty , language_mix , is_ambiguous , split
evaluation_runs	id , prompt_version_id , started_at , top1_accuracy , top3_accuracy , abstention_rate , false_confident_rate , results_json

1.3 Schedule Import — Your Contract with Kenneth

Import flow



Required and optional columns

Input column	Required	Example	Maps to
activity_id	Yes	MEC-L6-201	activities.activity_code
activity_name	Yes	Install Pump P-201	activities.name
wbs_code	Yes	PRJ.MEC.Z3.01	wbs_nodes.wbs_code
wbs_level	Prefer	6	wbs_nodes.level
planned_start	Yes	2026-08-20	baseline start
planned_finish	Yes	2026-08-24	baseline finish
duration_days	Prefer	5	calculated if absent
predecessor_ids	Yes for dependent tasks	CIV-L6-114;MEC-L6-198	join-table edges
dependency_type	Optional	FS	default FS
lag_days	Optional	0	default 0
discipline	Yes	Mechanical	matching filter
zone	Yes	Zone 3	matching filter
asset_tag	Prefer	P-201	exact-match boost
responsible_crew	Optional	Mechanical Crew B	context / dashboard
percent_complete	Optional	0	seed state; default 0
status	Optional	not_started	derived / defaulted

Validation rules — all of them

- no duplicate `activity_id` inside one project;
 - activity name, dates, discipline and zone are present;
 - finish is not earlier than start;
 - duration is positive and consistent enough to warn on mismatch;
 - every predecessor exists or is explicitly marked external;
 - no self-dependency;
 - **the dependency graph has no cycle;**
 - WBS parents exist or can be generated deterministically;
 - percent is within 0-100;
 - complete activities cannot have percent below 100;
 - actual dates present in an import are **staged for review**, never silently trusted;
 - **import commit is all-or-nothing.**
-

1.4 Actual START / FINISH and Progress Semantics

Event write rules

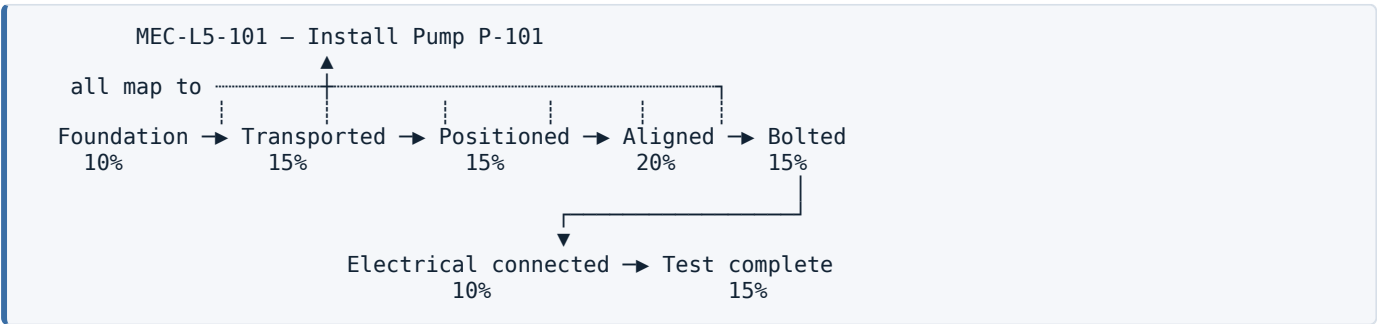
Extracted event	Approved write
START	set <code>actual_start</code> if absent; status → in_progress
FINISH	set <code>actual_finish</code> ; percent → 100; status → complete
PROGRESS	set approved percent or milestone-derived percent; ensure 0-100
BLOCKED	status → blocked; preserve percent; record reason
DELAY	create delay fact; update forecast input without claiming physical progress
RESUMED	status → in_progress; record resumed date
NO_EVENT	archive or request clarification; no schedule write

Conflict policy

- If a later approved actual start precedes an existing actual start, require explicit correction.
- A finish cannot precede start without reviewer correction.
- A lower reported percent than the approved percent is a **conflict**, not an automatic rollback.
- Duplicate source events use `client_event_id` and `review_decision_id` uniqueness to no-op safely.
- A rejected proposal remains available for learning but never changes actuals.
- Corrections create a new decision and audit entry; previous evidence stays visible.

1.5 Granularity Reconciliation Engine

The planned schedule may hold one activity; the field reports seven smaller events.



Derived-progress example

Sub-event	Weight	Approved state	Earned
Foundation available	10%	Complete	10%
Transported	15%	Complete	15%
Positioned	15%	Complete	15%
Aligned	20%	Complete	20%
Bolted	15%	Not complete	0%
Electrical connected	10%	Not complete	0%
Test complete	15%	Not complete	0%
Parent derived progress	100%	—	60%

Install Pump P-101

Planned progress [] 80%

Derived actual [] 60%

Variance [---- 20 percentage points ----]

The nine rules

1. Subtask templates are project/activity-specific and configurable.
2. Weights must total 100%; the MVP can seed them synthetically.
3. The first approved start-trigger milestone can propose the parent's actual start.
4. Parent progress equals the sum of earned approved milestone weights, unless an authorised reviewer overrides it **with a reason**.
5. Parent FINISH is proposed only when all `is_finish_required` milestones are complete.
6. Repeated reports for one milestone do not earn weight twice.
7. Out-of-sequence completion is allowed but flagged for review.
8. A field event can map directly to an L6 activity, to a subtask of an L5 activity, or remain unmatched.
9. Different discipline phrases may point to the same physical milestone through the synonyms dictionary.

Matching happens at two levels

Step A: Which parent activity / WBS branch does this concern?

Step B: Does the phrase describe a configured subtask milestone?

Step C: What parent progress/start/finish proposal follows from the approved milestones?

1.6 Deterministic Dependency and Delay Propagation

The MVP promise

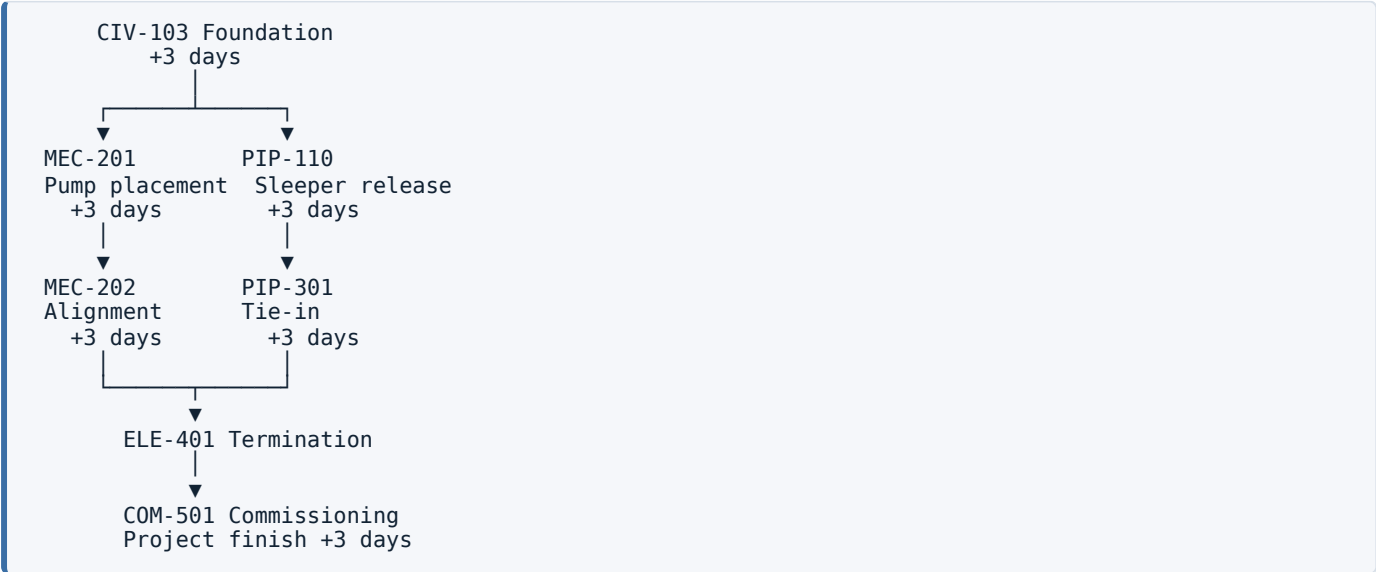
Finish-to-Start dependencies with lag and fixed working-day assumptions. Enough to demonstrate a real cascade without pretending to reproduce all of Primavera.

Forward pass

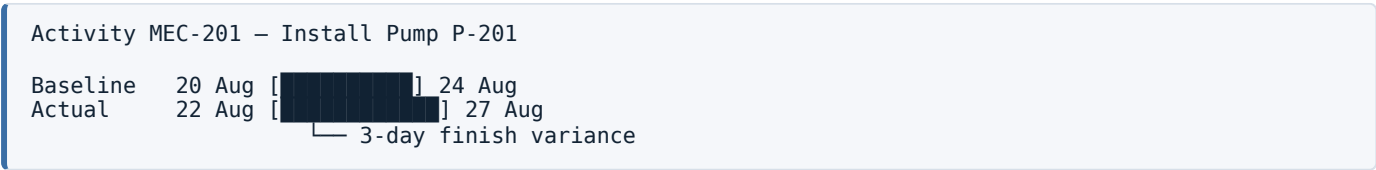
```
1. Load activities and FS dependency edges.
2. Detect cycles; refuse recalculation if any exist.
3. Topologically order the graph.
4. For each activity:
    predecessor_ready = max(predecessor actual/projected finish + lag)
    projected_start    = max(planned_start, predecessor_ready)
    remaining_days     = ceil(duration_days * (1 - approved_percent / 100))
    projected_finish   = actual_finish if complete
                        else projected_start + remaining_days
5. Preserve planned_start and planned_finish.
6. variance_days = projected_finish - planned_finish.
7. Store changed downstream activities and project-finish variance.
```

Define inclusive/exclusive date arithmetic once, write it down, and test it. A single off-by-one rule applied consistently is far safer than mixed assumptions. This is the bug this kind of generated code produces, and Akshaya's hand calculations exist to catch it.

Cascade shape



Plan vs actual bar



Delay without false progress

"Material not received; work stopped" creates a BLOCKED/DELAY proposal. After approval it moves the forecast or adds expected delay days **while percent complete stays unchanged**. This separates execution facts from forecast assumptions — and it is the beat that impresses judges most, because most tools cannot do it.

Required engine tests

- simple chain;
- two parallel branches;
- reconverging branches;
- lagged predecessor;
- completed predecessor using actual finish;
- partially complete successor;
- multiple predecessors choosing the latest ready date;
- cycle detection;
- no-op recalculation;
- repeated Confirm idempotency;
- Akshaya's five hand-calculated cascades.

1.7 The Transaction Boundary

Confirm and Correct must **fully** do all of this, or fully fail and show retry. No partial application.

```
1. insert review_decision
2. insert activity_actual / subtask_actual
3. update current activity state
4. recalculate downstream dates
5. insert impact and audit rows
```

Double-confirm must produce **one** actual and **one** schedule effect. The `review_decision_id` uniqueness constraint on `activity_actuals` is what enforces this — not UI button disabling.

1.8 Your API / Function Surface

Function / route	Input	Output / effect
POST /imports/schedule/preview	file + column map	normalised rows + validation errors
POST /imports/schedule/commit	preview token	transactional WBS/activity/dependency import
POST /field-updates	canonical source envelope + client ID	idempotent receipt
POST /field-updates/{id}/process	update ID	validated events + proposals
GET /proposals?status=pending	filters	review queue
POST /proposals/{id}/confirm	final values	atomic actual + recalculation + audit
POST /proposals/{id}/correct	selected target / final values / reason	corrected atomic write
POST /proposals/{id}/reject	reason	audit only
GET /activities/{id}	activity ID	schedule, milestones, evidence, history
GET /projects/{id}/dashboard	project ID	KPIs, variance, risks, review counts
POST /demo/reset	protected confirmation	restore frozen fixture

All mutation endpoints authenticate, authorise, validate project ownership, accept idempotency keys where relevant, and return structured errors.

1.9 Your Five Days

Day 1 • Sun 23 Aug — Freeze the foundation

Deliverable: approved schema + live Supabase project.

Design migrations. Build the dependencies join table. Write the import transaction. Draw the server/client boundary. Set RLS minimums. Publish the schema to **Kenneth** before he starts building the schedule — he cannot author to a schema that does not exist. Publish it to Soumyak too; he generates against it.

Spend 30 minutes with Kaushal: walk him through the schema, give him Supabase access, and hand him his first scoped task.

What you should have learned by end of Day 1: why a join table beats an array column for dependencies, and what breaks if baseline dates are mutable.

Concepts to hold: graph-friendly relational modelling; baseline immutability.

Day 2 · Mon 24 Aug — The safe pipeline

Deliverable: a safe `/parse-update` pipeline.

You build the `parse-update` Edge Function: schedule-context query, strict output validator, retry and failure path, secret handling. Soumyak owns the prompt inside it; you own everything around the prompt.

Your rule to enforce all week: **a bad LLM response becomes "needs clarification", never a crash and never a silent wrong write.** Never let a model-invented activity ID past the validator.

Kaushal's task today: the validator helper, or the schedule-context query. Both are self-contained and both are useful.

What you should have learned by end of Day 2: how to make a model's output safe before it touches a database.

Concepts to hold: JSON-mode / structured-output enforcement; server-side validation before any database touch; why the API key lives in the Edge Function and never in the browser bundle — a judge may check.

Day 3 · Tue 25 Aug — The engine (your hardest day)

Deliverable: deterministic recalculation engine + tests.

Generate and review the topological sort, cycle detection, FS forward pass, date utilities and idempotency contract. **Do nothing else today.** Soumyak and Akshaya handle all app-side work. Akshaya is hand-calculating five cascades in parallel; compare against her numbers, not against how the chart looks.

Give Kaushal something safely parallel today — a seed script, or the `delay_impacts` writer — so he is not blocked and you are not interrupted.

What you should have learned by end of Day 3: why date arithmetic is where generated code fails, and how an independent hand calculation catches it.

Concepts to hold: forward-pass scheduling / CPM basics; topological sort and cycle detection; why baseline and projected dates are stored separately; lag and lead.

Spend 30 minutes walking Akshaya and Soumyak through the recalculation logic. Neither becomes a replacement, but the area does not go dark if you are unavailable.

Day 4 · Wed 26 Aug — The write path

Deliverable: trustworthy write path.

Build the atomic proposal → decision → actual → recalculation transaction. Build the audit trail and the demo-reset backend. Transactional integrity is your call and nobody else's.

What you should have learned by end of Day 4: what "all of it or none of it" actually costs to implement, and why a unique constraint beats a disabled button.

Concepts to hold: transactional writes and idempotency.

Day 5 · Thu 27 Aug — Correctness only

Deliverable: a working end-to-end system, plus your half of the walkthrough.

Morning: P0 and P1 defects only, alongside Soumyak. You have authority to **refuse** any fix that risks the end-to-end path — a Day 5 change touching shared logic is often more dangerous than the bug it

fixes. Verify the reset works.

Afternoon: two clean end-to-end runs, then the walkthrough with Soumyak. Then the team retro.

What you should have learned by end of Day 5: which part of the backend you underestimated, and by how much. Write it down for the real build.

1.10 Your Checkpoints

DAY 1	Schema frozen and published to the team Dependencies join table live 40 tasks imported through the UI, not hand-entered Import commit is all-or-nothing Baseline dates preserved
DAY 2	No invented activity ID accepted by the server Malformed LLM output becomes a safe visible state API key absent from the frontend bundle
DAY 3	One 3-day mid-chain delay shifts 4+ downstream tasks exactly as Akshaya hand-calculated Baseline unchanged while projected dates move Cycle detected and rejected Date arithmetic rule written down and tested
DAY 4	Confirm is atomic – all five steps or none Double-confirm does not double-apply Reject changes no schedule facts Every actual traces to source and reviewer One-click demo reset restores known state
DAY 5	Zero P0 defects on the end-to-end path Reset verified working after two consecutive runs Your walkthrough segments delivered once Retro notes written

1.12 Your Half of the Walkthrough

Soumyak narrates; you take the technical half. Your segments:

Segment	What you show and say
Data foundation	The imported 40-task WBS, the dependency graph, and that the Gantt reads live from Postgres — not a hardcoded JSON file
The write path	A confirmed proposal becoming an <code>activity_actual</code> row, with the audit entry beside it
The cascade	The forward pass running, downstream projected dates moving, and the project-finish variance changing
Why not AI	That the scheduling maths is deterministic, tested code, and the model has no write path at all

Questions that come to you: schema design, transactions, idempotency, where the API key lives, cycle detection, what a double-confirm does, what breaks at scale.

If a question is about extraction, matching, confidence or the frontend, hand it to Soumyak. Handing off cleanly reads as a well-organised team.

1.11 What You Must Be Able to Explain in 60 Seconds

- Why the LLM is not in the scheduling path.
- How a proposal becomes an actual, and what prevents it becoming one twice.
- Where the API key lives and why the client never sees it.
- How a cycle is detected and what happens when one exists.
- Why baseline, actual and projected are three fields rather than one.

PART 2 — SOUMYAK

AI pipeline · frontend with Akshaya · offline capture · presenter

2.0 Your Mandate

You own **two things that meet in the middle**: the AI pipeline that turns messy field language into a structured proposal, and the frontend that a human uses to approve it. You are also the lead presenter.

What only you decide

- the wording, structure and few-shot content of the extraction and matcher prompt;
- how candidates are filtered and ranked before the model sees them;
- what the confidence bands mean and when the system abstains;
- how a requirement becomes a Lovable prompt, and when a generation attempt needs re-scoping smaller.

Your three irreplaceable outputs

1. **The extraction + L5/L6 matcher prompt** (Day 2). If informal text does not map to the right activity reliably, there is no product — just a Gantt chart with a text box next to it.
2. **The offline sync queue** that does not lose or duplicate updates (Day 3).
3. **The walkthrough** (Day 5).

Your boundaries. Arin owns the database, the Edge Function shell, the scheduling engine and the write path — you own the prompt *inside* his function, not the function. Akshaya owns design and the Gantt — you build the harder frontend wiring, she decides what it looks like. Neither of you should be waiting on the other for a whole screen.

2.1 One Adapter Contract for Every Source

Every ingestion path — PWA text, DPR, XLSX, CSV, voice, photo, simulated SMS — produces the same envelope. After this point, source-specific logic ends and your pipeline begins.

```
source_type
source_reference  (filename, sheet, row, device, simulated sender)
raw_text
reported_at / device_created_at
reporter identity if known
project identity
attachments
raw_payload
client_event_id
```

This single contract is why the WhatsApp and button-phone answers work in Q&A: swapping the transport is a new adapter, not a rewrite.

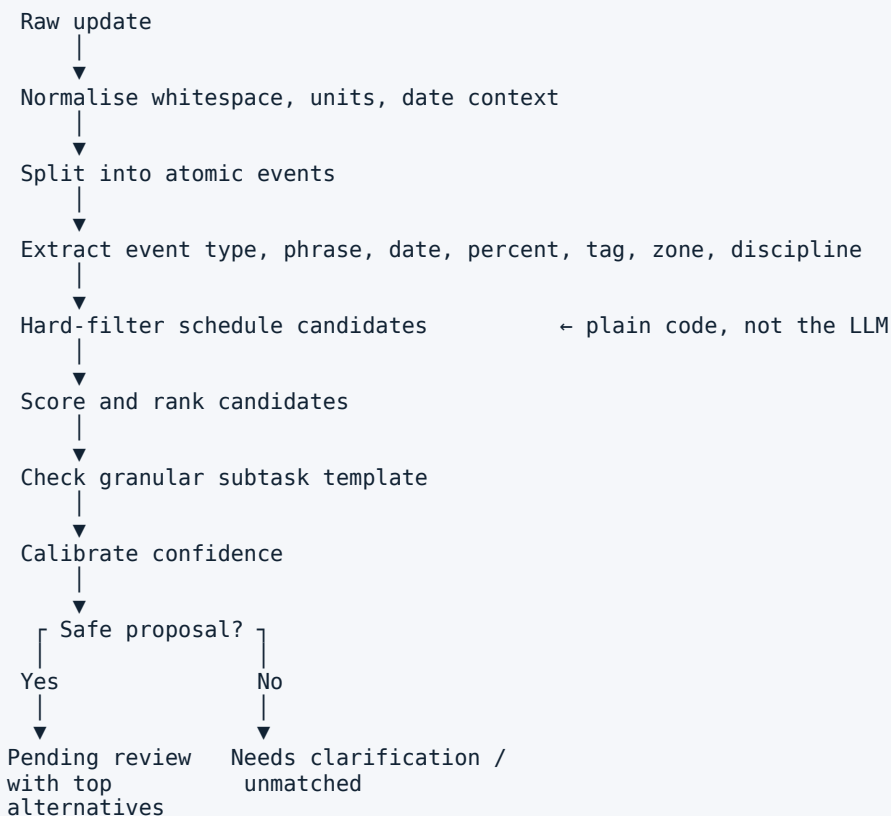
2.2 The Field-Update Taxonomy You Prompt Against

Tarun owns this vocabulary and owes it to you by **Day 2 morning**. Chase him for it — it is what makes the matcher domain-specific rather than a generic text matcher.

Meaning	Example English / site variants
START	started, commenced, shuru, mobilized, work opened
FINISH	done, completed, finished, khotom, ho gaya, closed
PROGRESS	60%, halfway, balance two joints, ongoing
BLOCKED	held, stopped, no access, material pending, permit pending
DELAY	late, slipped, rain delay, crew unavailable
RESUMED	restarted, resumed, clearance received
Civil	excavate, rebar, shuttering/formwork, cast/concrete, cure
Mechanical	erect, place, align, bolt, couple, test run
Piping	spool, fit-up, weld, NDT, hydrotest, reinstate
Electrical	cable lay, gland, terminate, megger, energize
Instrumentation	impulse line, loop check, calibrate, commission

It also covers zone aliases, asset-tag formats (`P-201` , `P201` , `Pump 201` , `pmp201`), contractor shorthand, and code-mixed Hindi-English / Assamese-English examples.

2.3 Processing Stages



Note where the model is and is not. Filtering is deterministic code. Ranking is model-assisted. Nothing here writes to the schedule.

2.4 Candidate Filtering — The Cheapest Accuracy Win

Before the model chooses anything, reduce the candidate list using:

1. project ID;
2. not-started / in-progress / blocked state appropriate to the event;
3. exact or normalised asset tag;
4. zone / location;
5. discipline;
6. plausible schedule window around the reported date;
7. WBS branch and known crew where available.

A FINISH report should not normally rank an activity already approved complete. A START report should not normally rank a distant completed activity. Exceptions stay available to a human through search and correction.

If top-1 is under 70%, the cause is almost always an unconstrained candidate list — not a cleverness problem in the prompt. Fix the filter before rewriting prose.

2.5 Hybrid Scoring Formula

Illustrative. Calibrate the weights on the test set; do not defend them as universal truth.

```
final_score =
  0.35 × semantic similarity
+ 0.20 × asset/tag match
+ 0.15 × zone match
+ 0.10 × discipline match
+ 0.10 × schedule chronology fit
+ 0.10 × activity-state fit
```

Worked comparison:

Candidate	Semantic	Asset	Zone	Discipline	Chronology	State	Final
MEC-L6-201 Install Pump P-201	.88	1.00	1.00	1.00	.90	.90	.94
MEC-L6-205 Install Pump P-205	.84	.20	1.00	1.00	.90	.90	.76
ELE-L6-401 Terminate P-201 feeder	.45	1.00	1.00	.10	.85	.90	.65

Look at what separates row 1 from row 2: semantic similarity alone would have ranked P-205 nearly as high. **The asset tag is the discriminator.** That is the single most useful thing to know about this dataset, and it is worth saying in the walkthrough.

2.6 Confidence Policy

Band	Score	Product behaviour
High	≥ 0.85	preselect top candidate; still requires Confirm
Medium	$0.65 - 0.849$	show top 3 prominently; correction/search equally easy
Low	< 0.65	no preselected write target; ask for clarification or manual selection
Hard conflict	any	force clarification if asset, zone or state contradicts the top match

Confidence is **not** the model's self-reported number. It combines measurable candidate signals and must be calibrated against correctness.

The metric that matters most is the false-confident rate — how often a High-band answer is wrong. Aim under 5%. A confident wrong match is worse than an abstention, because it is the one a reviewer might click through without thinking.

2.7 Strict Output Behaviour — Non-Negotiable

- Return **only** schema-valid JSON.
- Temperature near zero.
- **Never invent an activity ID** not present in the supplied candidates.
- Return 2–3 alternatives when possible.
- Split multi-activity messages into separate events.
- Set `needs_clarification: true` when information is insufficient.
- Resolve relative dates against report date and project timezone.
- Treat "pending" as blocked / not complete — **never** as finish.
- Validation failure becomes retry → clarification, not a crash.
- Record prompt / model / schema version for every extraction.

Canonical structured event JSON

```
{
  "source_update_id": "uuid",
  "events": [
    {
      "event_type": "FINISH",
      "activity_phrase": "Pump P-201 alignment",
      "reported_date": "2026-08-22",
      "percent_value": 100,
      "asset_tag": "P-201",
      "zone": "Zone 3",
      "discipline": "Mechanical",
      "delay_days": null,
      "delay_reason": null,
      "proposed_subtask": "ALIGN",
      "needs_clarification": false
    },
    {
      "event_type": "BLOCKED",
      "activity_phrase": "Electrical termination",
      "reported_date": "2026-08-22",
      "percent_value": null,
      "asset_tag": "P-201",
      "zone": "Zone 3",
      "discipline": "Electrical",
      "delay_days": null,
      "delay_reason": "pending",
      "proposed_subtask": null,
      "needs_clarification": true
    }
  ]
}
```

Arin's server validates enum values, dates, ranges, lengths, project identity and referenced activity IDs. Invalid output becomes a visible failure or clarification state — **never a silent database write.**

2.8 Photo and Voice — The Line You Must Not Cross

Photo evidence is **never** proof of completion. If it gets built at all, the vision model returns only observable facts:

GOOD:
"Concrete surface visible; formwork appears removed;
a board containing 'Zone 3' is readable."

FORBIDDEN:
"Foundation verified complete."

The literal prompt instruction is: *"describe what is visible; do not state whether work is complete."*

Voice, if implemented, is transcribed into `raw_text`; the audio stays as an attachment. The transcript goes through the same extraction and matching path. Voice is not a separate scheduling path.

Vision models are reliable at describing, reading text and identifying objects, and unreliable at judging completion, assessing quality or verifying claims. That distinction is the intellectual core of the design and the thing most worth saying out loud.

2.9 Field Capture and the Offline Queue

The capture screen

Ask only what a worker needs. Confirm capture, never interpretation.

```
REPORT SITE PROGRESS

What happened?
[ Pump P201 alignment done... ]

Date [Today ▼] Zone [Auto/▼]
[Add photo] [Record voice]*

[ SAVE UPDATE ]
● Offline capture available
```

State	Message shown
Offline	"Saved on this device — waiting for connection"
Online	"Received — processing"
Processed	"Proposal created — waiting for site-engineer review"

What works offline

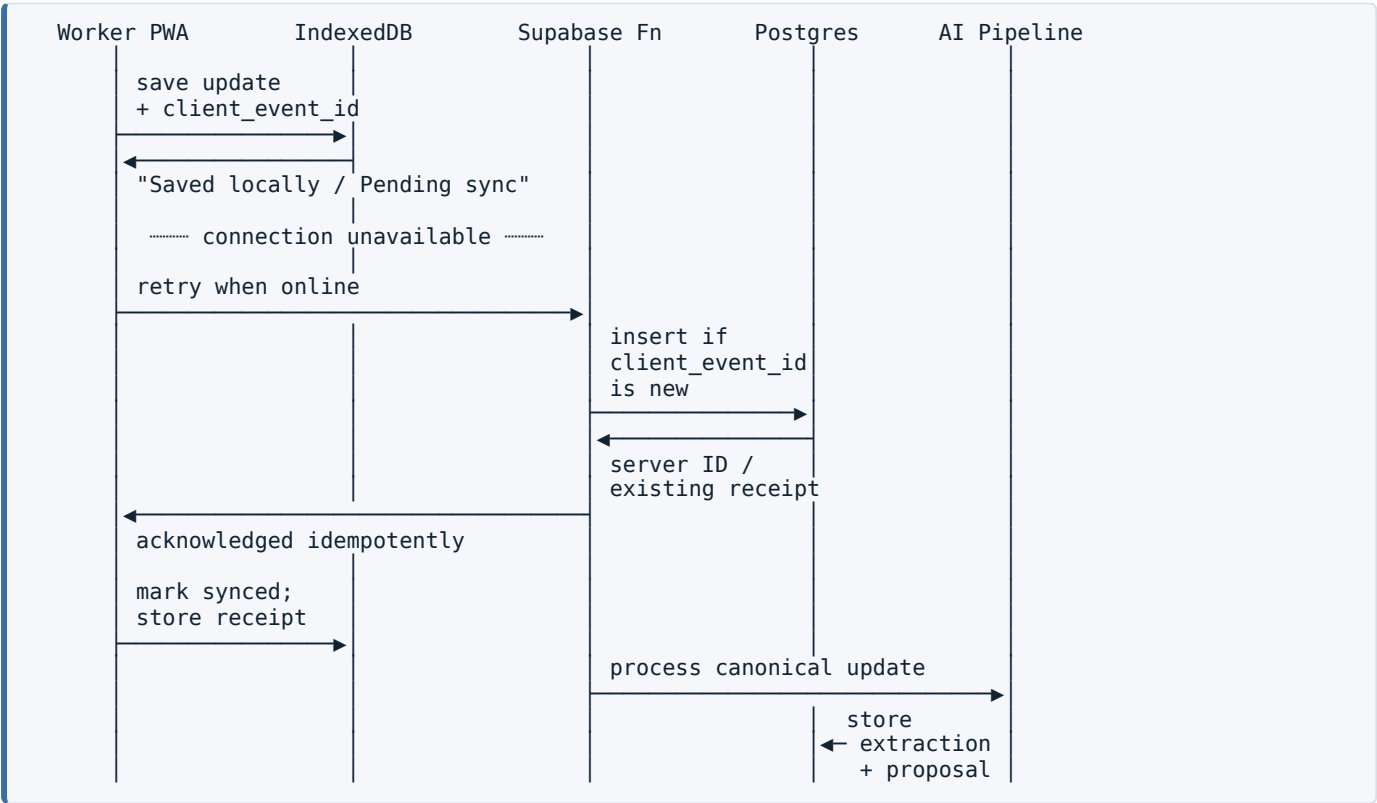
App shell and Field Mode load · user types an update with cached project/zone/crew context · update and attachment metadata save locally · pending items visible and editable until sync · manual retry available · cached recent submissions remain visible.

AI extraction, shared review and schedule updates happen **after** a server connection returns. Do not claim otherwise.

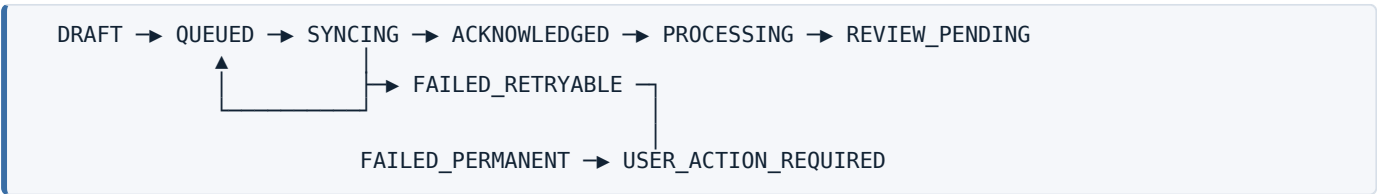
Client-only offline schema (IndexedDB)

Store	Key fields	Purpose
draft_updates	client_event_id , text, attachment metadata, device time	editable unsent draft
sync_queue	client_event_id , payload, retry_count, next_retry_at, last_error	durable outbox
cached_reference	project ID, recent activities, zones, crews, refreshed_at	offline selection context
sync_receipts	client_event_id , server update ID, acknowledged_at	prevents duplicate resubmission

Sync sequence



Queue state machine



Reliability rules

- **Never** delete a local item until the server acknowledges its `client_event_id`.
- Exponential backoff, with a visible manual retry.
- A network timeout may mean the server *did* receive it — retry with the same idempotency key.
- Attachment upload can retry separately while the text is safely received.

- Show exact status, last attempt and error in a Sync Center.
- Test airplane-mode capture, app reload, reconnection, duplicate retry, partial attachment failure.

2.10 DPR and Batch Ingestion

DPR input is either pasted report text or an XLSX/CSV daily report table. The adapter segments by row, bullet or sentence group. **Each segment becomes a separate `field_update`**, retaining filename and row reference. One segment can still yield multiple events.

DPR field	Use
report date	resolve "today" and set event context
contractor / crew	attribution and candidate context
discipline	candidate filter
area / zone	candidate filter
activity description	main extraction text
quantity / percent	progress proposal
status / remarks	START / FINISH / BLOCKED / DELAY meaning

2.11 Running the Evaluation

Kenneth builds the gold set and records results; you run the matcher against it and turn the output into a change. Your value is the diagnosis, not the number.

LOW VALUE:	"We're at 78% accuracy."
HIGH VALUE:	"78%. All 9 failures are ambiguous-piping cases where the message names no asset tag. Zone and discipline are correct in all 9. The filter is fine; the tie-break between three piping activities is not."

Failure category	Usually fixed by
Wrong zone	zone normalisation / filter
Wrong discipline	discipline filter, synonyms
Ambiguous phrasing	should have abstained — confidence calibration
Hallucinated ID	server validator (should already block it)
Date error	date-context in prompt
Event-type error	taxonomy few-shots
Granularity error	subtask template synonyms

Version every prompt you score. "82%" alone is worthless; "82% top-1 on 74 held-out cases, prompt v6" is evidence.

2.12 Edge Cases You Own

These are the states that break a live walkthrough. Test them yourself.

- confirming twice;
- rejecting after confirming;
- a proposal for an already-complete task;
- a photo with no message;
- a message matching nothing in the schedule;
- offline save followed by app reload;
- timeout, then retry with the same key;
- a batch import with one bad row;
- a malformed AI response;
- reset mid-session.

2.13 The Walkthrough — You Lead It

You narrate; Arin takes the technical half. Eight beats:

Beat	On screen	Who
1	Schedule row beside a messy field message — "plans are structured, reports are not"	You
2	Imported 40-task WBS and baseline Gantt, live from the database	Arin
3	Network off, submit, "Saved locally", reconnect, Sync Center acknowledges	You
4	One message splitting into two events, with confidence and alternatives	You
5	Confirm the clear event; correct the ambiguous one; the model has no write path	You
6	Progress moves, dates move, project finish shifts, audit row appears	Arin
7	KPIs, critical chain, activity history	You
8	Honest close — what is real, synthetic, simulated and roadmap	You

Beat 4 is the moment that proves it is more than keyword matching. Beat 6 is the moment the argument lands. Make sure the cascade is visible without scrolling.

Frozen messages, in order: clean START · granular FINISH with asset and zone · ambiguous wording that exposes alternatives · delay/block that moves project finish · optional photo-backed update described neutrally. Do not improvise demo text.

2.14 Your Five Days

Day 1 · Sun 23 Aug — The app exists

Deliverable: deployed URL reading live from Supabase; installable PWA shell.

Drive Lovable. Generate routing, the app shell, Supabase wiring, PWA manifest and service-worker scaffold. Build against Arin's frozen schema — do not let the tool invent its own structure. Akshaya starts her own screens in parallel; agree who takes which so you are not generating over each other.

Learn by end of day: what prompt shapes work when generating against an *existing* schema, and where the offline boundary sits in a PWA.

Day 2 · Mon 24 Aug — The matcher (your hardest day)

Deliverable: a versioned prompt scored against Kenneth's frozen gold set.

Constrain candidates first, then write the prompt. Run 6–8 disciplined iterations, scoring every one. Do not tune against cases you remember — that is how a held-out set stops being held out.

Learn by end of day: why retrieval quality bounds ranking quality, and how to make a model return the same shape every time.

Day 3 · Tue 25 Aug — Offline queue + matcher v2

Deliverable: offline capture that survives reload; matcher v2 with granularity.

Build the IndexedDB outbox, the Sync Center and the batch DPR/XLSX adapter. Then work Day 2's failure categories into specific prompt changes and add partial-progress extraction ("about 60% done" → `percent_value: 60`).

Keep app-side work off Arin — he is on the dependency engine all day.

Learn by end of day: idempotency from the client side, and why a timeout is not proof of failure.

Day 4 · Wed 26 Aug — Wire the trust gate

Deliverable: review queue and correction flow working end to end.

Wire the review queue to live AI results. Build the correction search and activity detail with Akshaya. Make processing errors visible and retryable rather than silent. Then run the edge-case list in §2.12 and log every defect with a severity.

Learn by end of day: what a human-in-the-loop gate costs in UI complexity, and which failure states you did not anticipate.

Day 5 · Thu 27 Aug — Integrate, run, present, retro

Deliverable: two clean end-to-end runs and a delivered walkthrough.

Morning: P0 and P1 fixes with Arin. Midday: freeze the fixture, prompt version and demo messages. Afternoon: two clean runs from reset, then the walkthrough with Arin, then the team retro.

Learn by end of day: which of your two areas — AI or frontend — actually took longer, and whether owning both was the right call for the real build.

2.15 Your Checkpoints

```
DAY 1 [ ] Deployed URL loads and reads from Supabase
      [ ] Built against Arin's schema, not an invented one
      [ ] PWA installable with an offline shell
      [ ] Screen split agreed with Akshaya

DAY 2 [ ] Candidate filter in place BEFORE prompt tuning
      [ ] 6-8 prompt iterations, each versioned and scored
      [ ] Top-1 accuracy measured on the frozen gold set
      [ ] Multi-event message produces multiple events
      [ ] Low-confidence case abstains rather than guessing
      [ ] No invented activity IDs survive validation

DAY 3 [ ] Airplane-mode update survives app reload
      [ ] Reconnection creates ONE server update after a
          duplicate retry
      [ ] DPR/XLSX rows reach canonical ingestion
      [ ] Partial-progress phrasing extracts a percent

DAY 4 [ ] Review queue wired to live AI results
      [ ] Correction changes the mapped activity and records it
      [ ] Every edge case in §2.12 tested and logged
      [ ] Processing errors visible and retryable

DAY 5 [ ] Two clean end-to-end runs after reset
      [ ] Prompt version frozen; matches any number quoted
      [ ] Walkthrough delivered once with Arin
      [ ] Retro notes written
```

2.16 What You Must Be Able to Explain in 60 Seconds

- The whole pipeline, in order, without notes.
- Why constraining candidates beats a cleverer prompt.
- Why the model returns alternatives and a clarification flag instead of always answering.
- What the accuracy number is, on how many held-out cases, at which prompt version.
- Why an offline update cannot be lost or duplicated.
- Why the cascade is deterministic code and not AI.
- What vision models are and are not reliable at.

PART 3 — AKSHAYA

Frontend and UI/UX with Soumyak · design lead · independent verifier of the scheduling maths

3.0 Your Mandate

You hold two authorities that most teams never combine: you decide what the product looks like, and you are the only person who independently checks whether Arin's engine is right. The second one matters more.

You build as well as design. You and Soumyak own the frontend together: you decide what a screen should look like and be, and you generate your own screens rather than queuing behind him. He takes the harder wiring — offline queue, batch ingestion, review-queue plumbing — and you take the dashboard, Gantt, activity detail and the review card's presentation. Agree the split on Day 1 so you are never generating over each other.

What only you decide

- the visual language: status colours, typography, hierarchy, projector legibility;
- how your own screens get generated and when they are good enough;
- whether a screen is usable in under five seconds;
- **whether the dependency engine's output is actually correct.**

Your one irreplaceable output: five hand-calculated cascade scenarios (Day 3) computed *before* you look at the engine's answer. This exists so the engine is never self-reviewed. Generated scheduling code produces off-by-one date errors that look completely plausible on a chart — you are the check that catches them.

The priority rule, written down so it is not a judgement call on the day. On Day 3 you are doing both verification and Gantt implementation. If both will not fit, **verification wins** and the visual work moves to Day 4. Getting the maths wrong is fatal; an unpolished chart on Day 3 is not.

3.1 The Verification Duty

How to do it

1. **Before** the engine exists, pick five scenarios from the synthetic schedule.
2. Compute each by hand — on paper or in Sheets — writing down every intermediate date.
3. Only then run the engine and compare.
4. Sign off each scenario as pass or fail, in writing, with your numbers next to the engine's.

The five scenarios must cover

#	Scenario	What it catches
1	Simple chain, one 3-day delay	basic forward-pass arithmetic
2	Two parallel branches from one predecessor	branch independence
3	Reconverging branches	the <code>max()</code> of multiple predecessors
4	Lagged predecessor	lag applied once, in the right direction
5	Partially complete successor	<code>remaining_days</code> scaling with percent

Also confirm: a completed predecessor uses its **actual** finish, not its projected one; a no-op recalculation changes nothing; and a deliberately introduced cycle is **rejected**, not silently ignored.

The worked form to use

SCENARIO 1
Trigger: CIV-103 finishes 3 days late (24 Aug instead of 21 Aug)

Activity	Planned finish	My calculation	Engine	Match
CIV-103	21 Aug	24 Aug (actual)	24 Aug	✓
MEC-201	26 Aug	29 Aug	29 Aug	✓
MEC-202	28 Aug	31 Aug	31 Aug	✓
ELE-401	30 Aug	2 Sep	2 Sep	✓
COM-501	2 Sep	5 Sep	5 Sep	✓

Project finish: 2 Sep → 5 Sep (+3 days)
Verdict: PASS – signed Akshaya, Day 3

Do not accept "it looks right on the chart." A one-day error is invisible on a Gantt bar and fatal in a judge's follow-up question.

3.2 Screens You Own

Navigation and modes

FIELD MODE

- Capture Update
- My Submissions
- Sync Center

CONTROL ROOM MODE

- Project Dashboard
- Review Queue
- Schedule / Gantt
- Activity Detail
- Import Schedule / DPR
- Audit Trail
- Historical Analytics
- Demo Reset (protected)

Screen-by-screen requirements

Screen	Primary user	Must contain	Key states
Project setup / import	Planner	file drop, header mapping, validation summary, row errors, dependency preview, commit	empty, parsing, invalid, ready, committed
Field capture	Worker / supervisor	large text box, date, optional zone/crew, photo/voice affordance, save	online, offline, saving, saved, error
My submissions	Reporter	original text, time, local/server status, proposal/review status	pending sync, processing, clarification, approved, rejected
Sync Center	Reporter	queued count, retry, last error, per-item status	offline, syncing, partial failure, all synced
Review queue	Site engineer	original evidence, extracted event, top match, alternatives, impact preview, decision buttons	loading, pending, low confidence, unmatched, stale
Correction drawer	Site engineer	activity search/filter, WBS context, event/date/percent edit, reason	no results, conflict, valid correction
Project dashboard	Manager	planned/actual %, baseline/projected finish, variance, at-risk tasks, pending reviews, recent activity	normal, delayed, no data, stale sync
Schedule / Gantt	Planner / manager	baseline + projected bars, progress, dependencies, today line, filters, click-through	initial, recalculating, error, delayed
Activity detail	Engineer / auditor	WBS path, dates, progress, granular milestones, evidence, history, dependencies	no evidence, conflicts, completed
Batch ingestion	Planner	DPR/Excel upload, row segmentation preview, processing results, unmatched rows	parsing, partial success, retry
Audit trail	Auditor / manager	filters, actor, source, before/after, decision, prompt/calculation version	empty, filtered, export-ready
Historical analytics	Manager / AI owner	match quality, corrections, durations, delay causes	insufficient data, demo data
Button-phone concept	Judge / demo only	simulated SMS text → canonical pipeline; clear simulation badge	optional
Demo reset	Team only	confirmation, restore fixture, progress, success/failure	protected, resetting, reset complete

Every screen needs an **empty, loading and error state**. A demo showing a spinner forever is a failed demo.

3.3 The Review Card — Your Most Important Screen

This is where the product's judgement becomes visible. It gets more design attention than anything else you build this week.

ORIGINAL UPDATE "Pump P201 alignment done. Electrical pending." Reporter: Mech Crew B • 22 Aug 2026 • Zone 3 [photo + neutral AI description, if present]	
PROPOSED EVENT 1 MEC-L6-201 Install Pump P-201 Subtask: Alignment complete • FINISH • 22 Aug Effect: parent progress 40% → 60% Alternatives: MEC-L6-205 • MEC-L6-209	HIGH 94%
PROPOSED EVENT 2 Electrical termination • BLOCKED Needs clarification: choose activity or add task code	LOW 58%
[CONFIRM] [CORRECT] [REJECT]	

The card must show **both levels** of the match — parent activity and milestone — and the effect if confirmed:

Field event: "P-101 alignment completed"
Parent: MEC-L5-101 Install Pump P-101
Milestone: ALIGN — Alignment complete (20% weight)
Effect: parent progress 40% → 60%

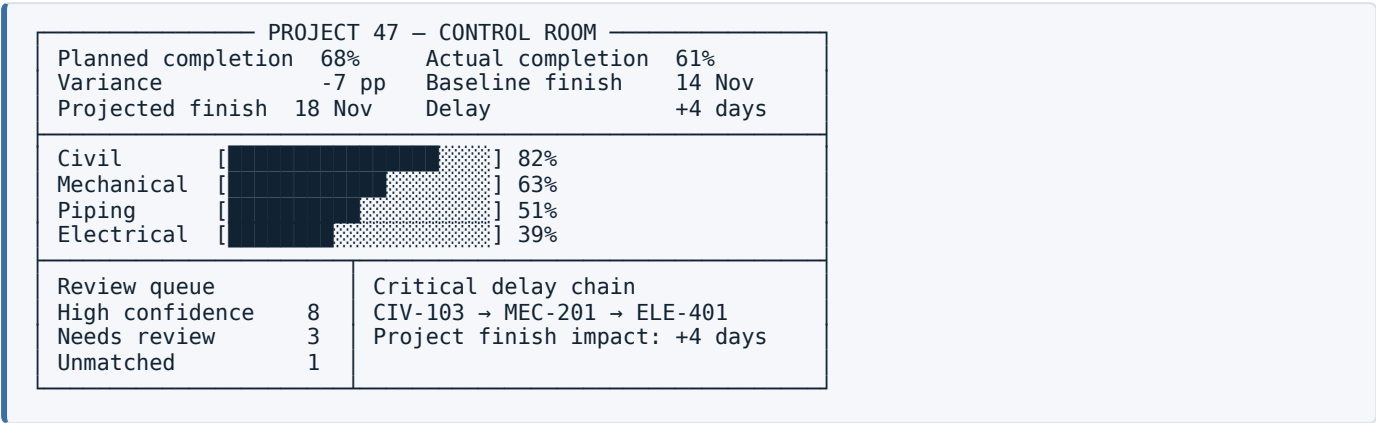
Decision behaviour

Action	Required behaviour
Confirm	Apply the exact proposed activity/event/date/percent in one transaction
Correct	Reviewer picks a search result or alternative and edits event/date/percent; original and correction both recorded
Reject	Require a reason category; write no actual
Clarify	Return a question to the reporter, or leave in the unmatched queue

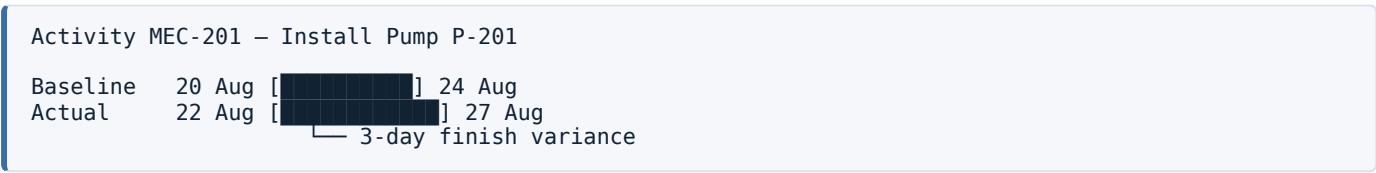
The five-second target

An authorised reviewer should understand and act on a normal **high-confidence** proposal in under five seconds. Uncertain proposals should deliberately take longer — make the safe action easy and the risky action require thought.

3.4 Dashboard Visual Contract



Plan vs actual bar



3.5 Design Rules

- **Colour plus text/icon/pattern — never colour alone.** Accessibility, and it is worth ten seconds in the pitch.
- Optimise for a **dim projector at four metres**: high contrast, large type, one focal point per screen.
- **The single most valuable layout decision this week**: side-by-side field input and schedule view, so cause and effect are visible without tab-switching.
- Confidence is shown as High/Medium/Low with a label and icon — never as a raw decimal in the primary view. Numeric detail lives in an expandable technical panel.
- Baseline sits behind, projected in front. Delay needs both colour and a non-colour marker.
- Errors must say what the user can do next.

3.6 Your Five Days

Day 1 · Sun 23 Aug — Visual direction + structural audit

Deliverable: design frame + zero-error structural audit.

Define status colours (not started / in progress / complete / at risk / blocked), typography, projector layout. One reference frame that the Gantt styling matches all week.

Then hand-audit Kenneth's schedule graph for cycles, orphans and impossible dates. Your coding background makes you the right person for this — nobody else will catch a subtle cycle by eye.

Learn by end of day: what a Gantt bar has to encode, and how to spot a cycle in a dependency list.

Concepts to hold: Gantt information hierarchy; how to spot a cycle in a dependency list.

Day 2 · Mon 24 Aug — Capture and proposal cards

Deliverable: approved review-card design.

Design the capture screen and proposal card. Confidence uses colour **plus** label and icon. Verify five-second scanability with someone who has not seen it.

Start generating your own screens rather than queuing behind Soumyak — he is on the matcher all day.

Learn by end of day: what a reviewer actually needs on screen to make a five-second decision, and what is decoration.

Concepts to hold: human-in-the-loop UX — enough context for a decision in under five seconds.

Day 3 · Tue 25 Aug — Verify first, then draw (your hardest day)

Deliverable: independent signed engine check.

Hand-calculate the five cascades **before** comparing to code. Then design the baseline/projected Gantt and verify the date rules.

If both do not fit: **verification wins**.

Learn by end of day: the inclusive/exclusive end-date convention, and why a one-day engine error is invisible on a chart.

Concepts to hold: how to hand-compute a schedule cascade accurately enough to catch a one-day error; why a Gantt shows baseline behind actual.

Day 4 · Wed 26 Aug — The product

Deliverable: a coherent, projector-ready product.

Own the Review Queue presentation, dashboard, Gantt states, task detail, and every empty/loading/error state — **designed and generated by you**. This is your highest-leverage day for how the thing feels, and the day Soumyak most needs the screens off his queue.

Learn by end of day: how many states a real screen has beyond the happy path.

Day 5 · Thu 27 Aug — Polish and hostility

Deliverable: presentation-ready screens.

End-to-end-path visual defects only. Do the projector check if you can get near one. Then take the hostile-question role: **Soumyak and Arin present**, so aim your meanest questions at both and watch whether they hand off cleanly rather than guessing.

Learn by end of day: which of your screens survived contact with a real run, and which you would design differently.

3.7 Your Checkpoints

```
DAY 1 [ ] Design tokens defined and shared
      [ ] Structural audit of the 40-task graph returns ZERO errors
      [ ] No cycles, orphans or impossible dates

DAY 2 [ ] Review-card design approved by the team
      [ ] Confidence readable without relying on colour alone
      [ ] Five-second scan verified with a fresh pair of eyes

DAY 3 [ ] Five cascades hand-calculated BEFORE seeing engine output
      [ ] Every scenario signed pass/fail with both sets of numbers
      [ ] Cycle deliberately introduced and correctly rejected
      [ ] Baseline unchanged while projected dates move

DAY 4 [ ] Your screens designed AND generated by you
      [ ] Every screen has empty, loading and error states
      [ ] Side-by-side field + schedule layout working
      [ ] Dashboard readable at projector resolution

DAY 5 [ ] End-to-end-path screens polished; nothing off-path touched
      [ ] Projector check done
      [ ] Hostile-question rounds run against Soumyak and Arin
      [ ] Retro notes written
```

3.8 What You Must Be Able to Explain in 60 Seconds

- How you verified the scheduling engine independently, and what you found.
- Why the review card shows alternatives and the effect-if-confirmed.
- Why colour is never the only signal.
- Why baseline sits behind projected rather than being overwritten.

PART 4 — KAUSHAL

Backend support under Arin · learning track

4.0 Read This First

You are the newest member and third in coding ability on this team, and that is fine — **this five-day run is a practice build**. Its purpose is that you finish the week understanding how this system works, having actually shipped parts of it, so that the real build is not the first time you see any of this.

Your part is deliberately shorter than Arin's or Soumyak's. That is by design, not a demotion. **A person who reliably delivers three small correct things is worth more than one who half-delivers something big.**

The rule that protects you and the project

```
ONE task at a time
SELF-CONTAINED  - one function or one endpoint, never a feature
CLEAR CONTRACT  - Arin states the input, output and failure shape
OFF the critical path - if yours is late, the walkthrough still works
REVIEWED        - Arin reads every line before it merges
```

If a task starts sprawling — you find yourself touching three files, or you cannot describe what "done" means in one sentence — **stop and go back to Arin to have it re-scoped smaller**. That is the correct move, not a failure. Sprawl is how a support task quietly becomes a blocker.

You are on the backend, not the frontend

Soumyak and Akshaya own the frontend entirely. Do not open Lovable. Your surface is Supabase: tables, queries, Edge Functions, and small server-side helpers, built with **Claude Code** and reviewed by Arin.

4.1 Your Onboarding Block — Day 1, First 90 Minutes

```
[ ] Read Part 0 of this guide, all of it
[ ] Read §1.2 of Arin's part - the database schema
[ ] Get Supabase project access from Arin
[ ] Open the table editor; look at `activities` and
    `activity_dependencies` until the join makes sense
[ ] Get Claude Code working against the repo
[ ] Ask Tarun for the one-sentence product description
    and say it back to him
[ ] Take your first scoped task from Arin
```

Do not write anything before you have read the schema. Every task you get will be about moving data between those tables. Guessing the shape wastes your time and Arin's review.

4.2 Your Mandate

You build **small, correct, reviewed pieces of the backend**, and you learn the whole system by touching its plumbing.

What you own

- whatever scoped task Arin has given you *right now* — one at a time;
- knowing your own task well enough to explain what it does and what breaks it;
- telling Arin early when something is going wrong, rather than late.

What you do not own

The dependency engine, the approval transaction, anything touching idempotency, the matcher prompt, and the frontend. Those are load-bearing and they belong to people with more headroom this week.

Your one irreplaceable contribution: being genuinely reliable on small things, so Arin's review time stays at 20–30 minutes a day and he stays on the engine.

4.3 The Kind of Task You Will Get

These are good-shaped tasks — self-contained, testable, useful, and safe if late.

Task	What it does	Why it is safe
CSV column normaliser	Trims, lowercases and maps header variants to canonical names	Pure function, no database writes
Validation helper	Checks a parsed row: dates ordered, percent 0–100, no self-dependency	Returns errors; writes nothing
Seed / reset script	Restores the fixture tables to a known state	Used only by the team
GET /activities/{id}	Returns one activity with its milestones and history	Read-only
Audit-log writer	Appends one row: actor, action, entity, before, after	Append-only, cannot corrupt
delay_impacts writer	Records old/new projected dates after a recalculation	Downstream of Arin's engine
Dashboard aggregate query	Counts pending proposals, at-risk tasks, completion percentages	Read-only

What a good task hand-off looks like

Arin should give you something in roughly this form. If he does not, ask for it — you are entitled to a clear contract.

```
TASK:      Validate one parsed schedule row
INPUT:     a row object with activity_id, name, dates, discipline,
           zone, predecessor_ids
OUTPUT:    an array of error strings; empty array means valid
RULES:     finish >= start; percent 0-100; discipline in the
           controlled list; predecessor_ids not containing own id
FAILURE:   never throw - return an error string instead
DONE WHEN: all seven fixture rows in tests/rows.json produce
           the expected error arrays
```

4.4 How to Work With Claude Code Without Creating Problems

You are not writing code by hand, but you are responsible for what lands. Five habits that make the difference:

- 1. Give it the contract, not the goal.** "Write a function that takes X and returns Y, and returns an error string instead of throwing" beats "make the import work."
- 2. One file at a time.** If Claude Code proposes changes across four files, that is a signal your task is too big. Narrow it.
- 3. Read the diff before accepting.** Every line. If you cannot say what a line does, ask Claude Code to explain that line before you accept it. This is the single highest-value habit you can build this week.
- 4. Check your own work in SQL.** After a write, open Supabase's SQL editor and `SELECT` the rows back. Do not report something as done because the code looked right.
- 5. Say when you do not understand it.** Handing Arin code you cannot explain wastes his review and teaches you nothing. "This works but I do not understand the date handling" is a useful sentence.

4.5 Topics You Specifically Need

Beyond the shared baseline in Part 0 §0.10, these are yours. None require you to write code from scratch — they let you direct and check it.

Topic	Why you need it
Reading a relational schema	Every task is "move data between these tables." Know which table a field lives in without asking
Foreign keys and join tables	Why <code>activity_dependencies</code> is a table and not a column, and what a join returns
SQL SELECT with WHERE and JOIN	So you can verify your own work instead of trusting the code
What a REST endpoint is	Method, path, request body, response body, status code
Pure functions vs side effects	Your safest tasks are pure; know why that makes them safe
Reading a diff	Additions, deletions, context. Non-negotiable before accepting anything
Append-only data	Why the audit log never updates or deletes, only inserts
Idempotency (concept only)	Why the same request twice must not apply twice — you will not implement it, but you must not break it
Environment variables and secrets	Why an API key lives server-side and never in the browser

Your Python is useful here even though the stack is TypeScript: it is what lets you *read* code fluently, which is the actual skill this week rewards.

4.6 Your Five Days

Day 1 · Sun 23 Aug — Onboard and ship one small thing

Deliverable: onboarding done, plus one completed scoped task.

Do the block in §4.1. Then take your first task from Arin — most likely the CSV column normaliser or the validation helper, since Kenneth's schedule import needs both.

Learn by end of day: the shape of the `activities` and `activity_dependencies` tables, and why the dependency join table exists.

Day 2 · Mon 24 Aug — Backend helpers

Deliverable: one or two completed tasks around the ingestion path.

Likely: the schedule-context query that feeds the matcher, or the validator that checks model output before it touches the database. Both are self-contained and both matter.

Learn by end of day: why a model's output has to be validated server-side before it is trusted, and what "never trust an ID the model produced" means in practice.

Day 3 · Tue 25 Aug — Stay parallel, stay off the engine

Deliverable: the seed/reset script, or the `delay_impacts` writer.

Arin is on the dependency engine all day and must not be interrupted. Take something that runs alongside it. The reset script is genuinely valuable — it is what lets the team run the walkthrough more than once.

Learn by end of day: what a dependency cascade is, by watching Arin's engine run and reading the rows it writes.

Day 4 · Wed 26 Aug — Audit and read paths

Deliverable: audit-log writer or the dashboard aggregate query.

Both are read-heavy or append-only — safe on the day the write path is being finished.

Learn by end of day: what an audit trail is for, and why "who changed what, when, on what evidence" is a product feature rather than bookkeeping.

Day 5 · Thu 27 Aug — Verify, watch, retro

Deliverable: your tasks verified working end to end; retro notes.

No new tasks. Verify that everything you built still works after integration. Run the full end-to-end flow yourself at least once — every member should have done this before Day 5 ends. Then watch the walkthrough and take notes for the retro.

Learn by end of day: where your pieces sit in the whole chain, and which one you underestimated.

4.7 Your Checkpoints

```
DAY 1 [ ] Onboarding block done
      [ ] Schema read BEFORE writing anything
      [ ] Supabase access working; Claude Code running
      [ ] One scoped task completed and reviewed by Arin

DAY 2 [ ] One or two tasks completed and reviewed
      [ ] You verified your own work with a SQL SELECT
      [ ] You can explain every line you handed over

DAY 3 [ ] A task delivered that did NOT interrupt Arin
      [ ] Reset script works, or delay_impacts rows appear
          correctly

DAY 4 [ ] Audit rows written with actor, action, before, after
      [ ] Nothing you built is on the critical path

DAY 5 [ ] Every task you built still works after integration
      [ ] You have run the full end-to-end flow yourself once
      [ ] Retro notes written - what took longer than you
          expected, and why
```

4.8 What You Must Be Able to Explain in 60 Seconds

- What each piece you built does, what its input and output are, and what breaks it.
- Where your pieces sit in the end-to-end chain.
- Why an API key never goes in the browser.
- Why the audit log only ever inserts.

- The one-sentence product description, worded the same way everyone else words it.

If you can do those five by Friday, this week worked — regardless of how much code came out of it.

PART 5 — TARUN

Team lead · scope controller · domain-knowledge owner · deck with Kenneth

5.0 Your Mandate

You hold **scope, taxonomy, evidence-policy and final cut authority**. Nobody else makes those calls. Your job is not to build the most — it is to stop the team building the wrong things, supply the domain credibility that makes the matcher look specialist rather than generic, and produce the deck with Kenneth.

You are not presenting. Soumyak and Arin deliver the walkthrough; they built it and can answer follow-ups. Your output is the *material* they present from — the product sentence, the taxonomy, the evidence policy, the demo script and the slides.

What only you decide

- what is in scope and what is cut, at any moment, without debate;
- the field-update taxonomy the whole AI pipeline is prompted against;
- the evidence and confirmation policy;
- what goes in the demo script and in what order;
- what the deck says and which claims survive fact-checking;
- final triage on Day 5 — every code change needs your sign-off, and the default answer is no.

Your three irreplaceable outputs: the written Definition of Done and cut list (Day 1), the field taxonomy plus evidence policy (Days 2–3), and the deck with Kenneth (Days 4–5).

The warning, stated plainly. Your domain work — taxonomy, evidence policy, Q&A — *feels* like paperwork next to visible building, and you will be tempted to deprioritise it. It is not paperwork. It is what makes the matcher domain-specific, what gives you credibility with an EPC judge, and what your Q&A preparation is built from. Give it real calendar slots on Days 2 and 3 or it will quietly disappear.

5.1 Day 1 — The Scope Contract

Write these three documents on Day 1 morning and get every member to acknowledge them in writing.

1. The product sentence

The single sentence every member must be able to say identically:

AI sits between field execution and project control to propose trustworthy schedule-linked actuals; humans approve them, and deterministic code calculates their impact.

2. The Definition of Done

The MVP is done when the full chain in §0.2 runs without anyone editing the database by hand. Not "the screens work." The acceptance unit is the **end-to-end data journey**.

3. The not-doing list

NOT BUILDING IN FIVE DAYS:

- x direct live Primavera P6 / MS Project API integration
- x real WhatsApp Business deployment
- x real SMS / IVR / USSD telecom integration
- x native Android / iOS app
- x full offline AI
- x autonomous visual verification from photos
- x full RBAC, SSO, multi-tenant controls
- x complete CPM (SS / FF / SF, leads, resource levelling, calendars)
- x BIM / 3D digital twin
- x large-scale vector retrieval over tens of thousands of activities
- x production security certification

WE BUILD ONE CONVINCING VERTICAL SLICE.

5.2 The Field-Update Taxonomy — Your Day 2 Deliverable

This is the controlled vocabulary Soumyak prompts against, Kenneth builds test cases from, and Akshaya writes UI copy from. Deliver it by Day 2 morning; three people are blocked without it.

Meaning	Example English / site variants
START	started, commenced, shuru, mobilized, work opened
FINISH	done, completed, finished, khotom, ho gaya, closed
PROGRESS	60%, halfway, balance two joints, ongoing
BLOCKED	held, stopped, no access, material pending, permit pending
DELAY	late, slipped, rain delay, crew unavailable
RESUMED	restarted, resumed, clearance received
Civil	excavate, rebar, shuttering/formwork, cast/concrete, cure
Mechanical	erect, place, align, bolt, couple, test run
Piping	spool, fit-up, weld, NDT, hydrotest, reinstate
Electrical	cable lay, gland, terminate, megger, energize
Instrumentation	impulse line, loop check, calibrate, commission

The taxonomy must also cover:

- **zone aliases** — how the same location gets named three different ways;
- **asset-tag formats** — P-201 , P201 , Pump 201 , pmp201 ;
- **contractor shorthand** — the abbreviations a specific crew habitually uses;

- **code-mixed examples** — Hindi-English and Assamese-English phrasing as it actually appears.

This document is also a slide in the deck and an answer in the Q&A. It is the single clearest piece of evidence that you understood the domain rather than building a generic text-matcher.

5.3 The Evidence & Confirmation Policy — Your Day 3 Deliverable

This turns "we added a Confirm button" into "we designed a governance layer."

Suggested evidence minimums

Event	Suggested evidence / check
Civil visible completion	reporter + location; photo helpful
Hydrotest completion	pressure and duration reference; photo alone insufficient
Electrical energisation	permit / test reference; explicit authorised confirmation
Partial progress	quantity or percent basis where possible
Delay / blocked	reason and expected resolution date where known

Authorisation

The MVP can use one authenticated site-engineer role, but the UI and audit must make the authorised decision **explicit** — who decided, when, on what evidence.

What the policy must state

1. who is authorised to confirm a status change;
 2. what evidence is sufficient, per discipline;
 3. what must be recorded for every decision;
 4. that corrections append rather than overwrite;
 5. that rejected proposals are retained for learning but never change actuals.
-

5.4 Risk Register — Yours to Watch

Risk	Early warning	Prevention	Five-day fallback
Matcher accuracy too low	top-1 <70% on Day 2	gold set first; filter by zone/discipline/tag/state	reduce demo schedule to 20 distinct tasks; emphasise human gate
Dependency engine wrong	mismatch with Akshaya's hand results	fixed date semantics; topological tests	FS-only, fixed durations, no critical-path highlighting
Granularity becomes vague	subtasks have no weights or rules	configure one deep, credible parent example	support one template family extremely well
Offline sync duplicates	timeout retry creates two rows	client UUID + unique constraint + receipts	manual Sync button using the same idempotency key
XLSX variation breaks import	headers/dates parse inconsistently	header mapping + preview + fixtures	demonstrate CSV and one frozen XLSX template
AI response malformed	JSON parser errors	strict schema + validation + retry	safe clarification; precomputed demo result only if labelled
API / network failure at venue	slow or no response	quota check, hotspot, cached assets	fallback video/screens; continue narrating
Real dataset unavailable	schedule seems generic	synthetic but domain-reviewed fixtures	state the limitation first; show construction method
Photo overclaim	model says "complete"	neutral-description prompt and UI copy	remove AI caption; show raw evidence only
Your domain work slips	generic prompts and weak Q&A	calendar blocks on Days 1–3	cut optional visuals; protect taxonomy and evidence policy
Akshaya overloaded	engine verification delayed	verification takes priority	move polish to Day 4; simplify Gantt styling
Soumyak unavailable	AI pipeline and frontend both stall	Akshaya generates her own screens; Arin can cover the Edge Function	freeze at last tested version; cut features; show what exists
Kaushal's tasks sprawl	he is touching three files for one task	Arin scopes one self-contained task at a time, off the critical path	Arin absorbs the task; Kaushal moves to verification and test execution
Arin unavailable	correctness authority and backend gone	cross-walk recalculation logic to Akshaya and Soumyak on Day 3	freeze the engine at its last verified state; no further schedule changes

Risk	Early warning	Prevention	Five-day fallback
You over-committed	owning scope AND taxonomy AND the deck	Kenneth builds the slides; you supply only the argument	drop optional deck polish; protect the taxonomy and evidence policy
Scope creep	suggestions for WhatsApp, native app, full CPM	visible cut list; your final say	remove Tier 2 immediately
Security questions	client exposes secrets or data	server-only key, minimum auth, audit	explain prototype boundary and on-prem roadmap

Cut order if behind — cut from the bottom upward, no mid-crisis debate

1. Second demo scenario
2. Rich historical charts
3. Real-time photo description
4. Voice input
5. Simulated SMS screen
6. Extra dashboard polish
7. Multi-format DPR beyond one XLSX + pasted text
8. Advanced Gantt decoration / critical-path highlight
9. Granularity families beyond the one strongest example

NEVER CUT:

- schedule import and real database
- extraction + L5/L6 matching
- human Confirm / Correct / Reject gate
- transactional actual write + audit
- deterministic dependency cascade
- offline-safe capture
- reset and fallback

5.5 Daily Stand-Up Template

Run this at the start and end of every day. Ten minutes, standing up, no laptops.

DAY:
Current end-to-end checkpoint:

GREEN – working and verified:
1.
2.

AMBER – working but unverified / fragile:
1.
2.

RED – blocking the pipeline:
1.
2.

Today's single critical path:
Owner:
Independent verifier:
Acceptance test:

Features proposed today but cut/deferred:
Tarun decision:

End-of-day evidence link:
Reset status:

5.6 Your Five Days

Day 1 · Sun 23 Aug — Freeze the scope

Deliverable: a signed scope contract.

Product sentence, Definition of Done, not-doing list, domain assumptions. Get written acknowledgement from all five others. Enforce no feature drift from this point.

Concepts to hold: a tight 30-second product definition; how to defend scope without sounding obstructive.

Day 2 · Mon 24 Aug — The domain pack

Deliverable: the taxonomy used in prompt, tests and UI copy.

Field taxonomy, synonyms, evidence language, zone and asset conventions. Soumyak and Kenneth are both blocked until this exists — deliver it in the morning, not the evening.

Learn by end of day: how much of matching accuracy is domain vocabulary rather than model quality.

Concepts to hold: EPC vocabulary; WBS meaning; enough of the matcher's behaviour to explain it in 45 seconds without saying "it uses AI."

Day 3 · Tue 25 Aug — Governance and the midday cut

Deliverable: governance policy + a scope decision.

Finalise the evidence and confirmation policy. Then, at noon, run an honest scope review against this guide and **decide what gets cut**. Not at the end of the week — now, while cutting is still cheap.

Concepts to hold: audit trails, and why regulated / PSU environments demand attributable changes.

Day 4 · Wed 26 Aug — Freeze the feature list

Deliverable: frozen feature list + demo script.

At midday, decide whether any optional work proceeds. Write the microcopy with Akshaya and the final demo sequence beat by beat, with timings. **Nothing new gets built after tonight.**

Concepts to hold: demo-script craft — open with the pain in one sentence, working system on screen within 90 seconds, one clean surprise (the cascade), close on differentiation. No feature tours.

Day 5 · Thu 27 Aug — Triage, deck, retro

Deliverable: finished deck + the written retrospective.

Morning: triage authority. Every code change needs your explicit sign-off and the default answer is no. Rank defects P0/P1/P2 with Soumyak; anything cosmetic off the end-to-end path is P2 and probably never gets fixed — say that out loud so nobody burns the day on it.

Afternoon: finish the deck with Kenneth. Watch Soumyak and Arin's walkthrough and give feedback on the narrative — you wrote the script, so you are the person who notices when a beat is missing.

End of day: run the retrospective. This is the actual deliverable of a practice week. Get the five questions from Part 0 answered honestly, written down, and kept. A missed gate with a recorded cause is worth more than a hit gate nobody understands.

Learn by end of day: which parts of your scope contract the team ignored, and why — that tells you how to write the real one.

5.7 Your Checkpoints

- DAY 1
 - ☐ Scope contract written and acknowledged by all six
 - ☐ Not-doing list visible where the team works
 - ☐ Product sentence agreed word for word
- DAY 2
 - ☐ Taxonomy delivered by MORNING, not evening
 - ☐ Soumyak using it in few-shot examples
 - ☐ Kenneth using it to write test cases
- DAY 3
 - ☐ Evidence and confirmation policy written
 - ☐ Noon scope review held; cut decision recorded
 - ☐ Stand-up run at start and end of day
- DAY 4
 - ☐ Feature list frozen in writing
 - ☐ Demo script written beat by beat with timings
 - ☐ Speaker assignment decided
- DAY 5
 - ☐ Every code change signed off individually
 - ☐ Deck finished and fact-checked with Kenneth
 - ☐ Walkthrough watched; narrative feedback given
 - ☐ RETROSPECTIVE run and written down
 - ☐ Q&A handoffs agreed - who answers what

5.8 What You Must Be Able to Explain in 60 Seconds

- The product sentence, verbatim.
- What is real, what is synthetic, what is simulated, and what is roadmap — without hesitating.

- Why the human gate exists and what it prevents.
- Why the team built a taxonomy rather than a generic matcher.
- What you cut, and why cutting it was correct.
- Which of your five teammates answers each kind of technical question:
architecture and the write path → Arin · extraction, matching and
accuracy → Soumyak · frontend and Gantt → Akshaya · data and test
method → Kenneth.

PART 6 — KENNETH

Synthetic data · evaluation dataset · test execution · deck with Tarun

6.0 Your Mandate

You are the **evidence, dataset and testing-support owner**. Almost every measurable claim the team makes on stage traces back to something you built. The accuracy number is only real because your gold set was built honestly and before the prompt.

What only you decide

- what the synthetic schedule contains and whether it is plausible;
- what goes in the evaluation set and what the gold label is;
- whether a number in the deck is verified or gets cut.

Your two irreplaceable outputs: the 40-task synthetic schedule (Day 1) and the 60–80-case gold-labelled test set (Day 2). Nothing downstream works without the first, and the team's central technical claim does not exist without the second.

Why order matters more than volume. The test set must exist **before** Soumyak tunes the prompt. If it is built afterwards, or edited while tuning, the accuracy number stops being evidence and becomes a claim. Guard that ordering — it is the whole point of your Day 2.

You also own the deck with Tarun. He supplies the argument and the wording; you build the slides and fact-check every number in them. Soumyak and Arin present from your material — so if a claim cannot be traced to a source, it comes out before it ever reaches a slide.

6.1 The Synthetic Schedule — Day 1

Build in Google Sheets, to the schema Arin publishes on Day 1 morning. Export as both CSV and XLSX.

Quality bar — all of it must be true

- **40 realistic** well-site / pipeline EPC activities;
- disciplines: civil, mechanical, piping, electrical, instrumentation, pre-commissioning;
- **at least six dependency levels** deep;
- **2-3 parallel branches that reconverge** — a straight line looks fake to anyone who has used Primavera;
- meaningful asset tags, zones, crews and WBS codes;
- **at least one larger activity with 5-7 granular field subtasks** (this becomes the granularity demo);
- no cycles, orphans, impossible dates, or generic names such as "Task 14."

Columns to produce

Column	Required	Example
activity_id	Yes	MEC-L6-201
activity_name	Yes	Install Pump P-201
wbs_code	Yes	PRJ.MEC.Z3.01
wbs_level	Prefer	6
planned_start	Yes	2026-08-20
planned_finish	Yes	2026-08-24
duration_days	Prefer	5
predecessor_ids	Yes for dependent tasks	CIV-L6-114;MEC-L6-198
dependency_type	Optional	FS
lag_days	Optional	0
discipline	Yes	Mechanical
zone	Yes	Zone 3
asset_tag	Prefer	P-201
responsible_crew	Optional	Mechanical Crew B
percent_complete	Optional	0
status	Optional	not_started

Naming realism

Use AI to draft activity names, then read every one and ask whether an EPC engineer would recognise it.

GOOD:

Skid Erection & Alignment
Hydrotest of Flowline Section 2
Terminate Pump P-201 Feeder
Rebar and Formwork – Zone 3 Foundations

BAD:

Task 14
Mechanical Work
Phase 2 Activities

Akshaya audits your graph for cycles and orphans on Day 1 evening. Expect to fix things.

Learn by end of day: WBS structure, Finish-to-Start dependencies, and what makes an EPC schedule look plausible rather than invented.

6.2 The Evaluation Dataset — Day 2

60–80 gold-labelled messages, written against the schedule you built and Tarun's taxonomy.

Coverage dimensions — every one must appear

Language:	clean English / abbreviations / Hindi-English / Assamese-English
Clarity:	clean / incomplete / ambiguous / negative
Event:	START / FINISH / PROGRESS / BLOCKED / DELAY / RESUMED
Granularity:	direct L6 / parent + subtask / cross-discipline synonym
Context:	correct asset / missing asset / conflicting asset / zone only
Input:	single event / multiple events / report row / noisy text

At least 15 must be genuinely hard. Examples of what "hard" means:

- ambiguous by design — "piping work finished" when three piping activities exist;
- conflicting context — an asset tag that belongs to a different zone than the one named;
- a negative — a message about something not in the schedule at all;
- a delay report that must **not** be read as progress;
- a multi-event message where one event is clear and the other is not.

Gold labels

Every case carries: `message`, `gold_activity_id`, `gold_event_type`, `gold_date`, `difficulty`, `language_mix`, `is_ambiguous`, `split`.

Hand-check every gold label. A wrong gold label makes the whole measurement wrong in a way nobody will notice until a judge does.

Keep a **true held-out split**. If the prompt is repeatedly edited against held-out failures, either create a new split or record the limitation honestly.

The gold-set sheet you hand to Soumyak

Agree the exact column names with him before writing rows. A workable shape:

Column	
case_id	stable identifier
message	the raw field text
gold_activity_id	correct L5/L6 activity
gold_event_type	START / FINISH / PROGRESS / BLOCKED / DELAY / RESUMED / NO_EVENT
gold_date	expected resolved date
gold_subtask	milestone code, where applicable
difficulty	easy / medium / hard
language_mix	english / abbreviated / hindi-en / assamese-en
is_ambiguous	true where abstention is the correct answer
split	train / heldout

He runs the matcher against it and fills in predicted label, confidence, band and pass/fail per case. You and he then categorise every failure together.

6.3 Metrics You Report

Metric	Five-day target	Failing means
Top-1 accuracy on held-out set	≥80%	matcher claim too weak
Top-3 accuracy	≥95% target	correction UI will not save reviewer time
False-confident rate on wrong matches	<5% target	confidence calibration is unsafe
Event-type exact match	≥90% target	START/FINISH semantics unreliable
Asset / zone extraction	≥90% target	candidate filter loses signal
Abstention on ambiguous cases	≥80% target	model guesses too often
Granularity: gold parent + subtask	≥85% on demo fixture	central differentiator unproven

Report the number **with the prompt version and the date**. "82%" alone is worthless; "82% top-1 on the 74-case held-out set, prompt v6, 24 Aug" is evidence.

6.4 Fixtures and Test Execution — Day 3

Build the fixture library the engine and import tests run against.

- **granular milestone fixtures** — approved-like events for the 5-7 subtask parent;

- **delay cases** — a mid-chain 3-day slip with known downstream effect;
- **10+ import files** — valid, plus deliberately broken ones: duplicate activity ID, missing predecessor, self-dependency, a cycle, finish-before-start, percent out of range, bad date format;
- **optional plausible site photos**, 10–12, matched to specific activities.

Then execute Arin's engine test scripts and record results. You run the scripts; Akshaya independently hand-calculates. Two different checks, deliberately.

Learn by end of day: what reproducible test execution means — same fixture, same steps, recorded outcome.

6.5 The Deck — Days 4-5

Structure

1. The problem – plan vs execution language gap
 2. Who it hurts and how
 3. What exists already (OpenSpace, Buildots, Track3D, ALICE, Xpedeon) and why we are different
 4. What we built – architecture, plain
 5. The three-part safety architecture
 6. Live demo
 7. Measured results – accuracy, with method
 8. Evidence and governance policy
 9. HONEST LIMITATIONS – real / synthetic / simulated / roadmap
 10. Business model
 11. What production deployment adds

The honest-limitations slide

Put it **before** the business model, not hidden at the end. Naming a limitation first defuses the question; being caught on it does the opposite.

Component	Status in the demo
Project schedule	Synthetic but realistic OIL-style EPC schedule
Primavera integration	CSV/XLSX import — not live API integration
WhatsApp / SMS	Simulated input format; real telecom integration is roadmap
AI task matching	Real. Live LLM call, measured on a held-out test set
Photo handling	Real upload + neutral AI description as human-reviewable evidence
Dependency recalculation	Real. Deterministic FS forward pass
Human confirmation	Real and mandatory — no status change without it

The rule on numbers

Every number in the deck must match the frozen evaluation or database result. If you cannot trace a figure to its source, cut the figure. You are the last line of defence on this — fact-check every one on Day 5.

6.6 Fallback Assets — Day 5

Venue wifi fails, APIs rate-limit, laptops sleep. A team that keeps pitching through a technical failure looks composed; a team that cannot look finished.

- [] Full screen-capture video of one clean end-to-end run
- [] Static screenshots of every key screen, in the deck as backup slides
- [] Offline copy of the deck on at least TWO devices
- [] Video also on a second device
- [] Frozen demo messages and photos stored and labelled

6.7 Your Five Days

Day 1 · Sun 23 Aug — The schedule

Deliverable: validated CSV and XLSX fixtures.

Build the 40-task schedule: WBS, zones, disciplines, asset tags, 6-level chain, parallel branches. Full day of careful work — treat it as a real deliverable, because everything downstream depends on it.

Concepts to hold: WBS structure and why codes are hierarchical; Finish-to-Start dependencies; float/slack basics; the rough sequence of an oil-field surface facility build.

Day 2 · Mon 24 Aug — The gold set

Deliverable: frozen evaluation dataset + scorecard.

Agree the CSV format with Kaushal first thing. Then build and hand-check the 60–80 case gold set **before** he starts tuning. His harness computes top-1, top-3, abstention and false-confidence from your file.

Concepts to hold: what a labelled evaluation set is; why building it first prevents unconsciously tuning to remembered examples; the difference between accuracy and "it worked when I tried it."

Day 3 · Tue 25 Aug — Fixtures and evidence

Deliverable: granularity and scheduling evidence pack.

Granular milestone fixtures, delay cases, 10+ import files including broken ones, optional site photos. Execute the engine test scripts and record results.

Concepts to hold: reproducible test execution — same fixture, same steps, recorded outcome.

Day 4 · Wed 26 Aug — Regression and deck

Deliverable: test evidence + near-final deck.

Load the demo fixtures. Execute the regression matrix (Part 7, F01–F20). Build the deck with Tarun: he supplies the argument and the wording, you build the slides. Use real measured numbers only.

Learn by end of day: how many things quietly break when other people's work is integrated.

Concepts to hold: how a technical pitch deck is structured; why naming limitations first defuses the question.

Day 5 · Thu 27 Aug — Freeze, verify, operate

Deliverable: complete evidence and fallback package.

Freeze the demo data. Finish and fact-check the deck with Tarun — every claim traced to a source, anything unverifiable removed. Compile the Q&A sheet.

Soumyak and Arin deliver the walkthrough, so your job is the material they present from, plus recording what actually happened for the retro.

Learn by end of day: which of your numbers survived fact-checking and which did not — that ratio tells you how careful to be in the real build.

6.8 Your Checkpoints

- DAY 1

☐

☐

40 activities, 6+ dependency levels, 2-3 reconverging branches

☐

☐

☐

☐

☐

One parent activity with 5-7 granular subtasks

Zero generic names

CSV and XLSX both export cleanly

Akshaya's structural audit passes
- DAY 2

☐

☐

☐

☐

☐

☐

☐

☐

Sheet format agreed with Soumyak BEFORE writing rows

60-80 cases written BEFORE prompt tuning began

Every gold label hand-checked

All six coverage dimensions represented

At least 15 genuinely hard cases

Scorecard produces top-1, top-3, abstention, false-confidence
- DAY 3

☐

☐

☐

10+ import fixtures including deliberately broken ones

Granular milestone and delay fixtures ready

Engine test scripts executed and results recorded
- DAY 4

☐

☐

Full F01-F20 regression matrix executed

Deck built with real measured numbers only
- DAY 5

☐

☐

☐

☐

Every deck number traced to its source

Fallback video recorded

Deck and video on two devices

Demo data frozen and labelled

6.9 What You Must Be Able to Explain in 60 Seconds

- How the synthetic schedule was constructed and why it is representative.
- How the test set was built, and why building it first matters.
- What each metric means and what its failure would imply.
- What in the demo is real, what is synthetic, and what is simulated.

PART 7 — SHARED REFERENCE

Everyone reads this before Day 4 ends

7.1 End-to-End User Flows

A planner imports a schedule

1. Select project and upload CSV/XLSX.
2. Map headers; the system remembers the mapping for the session.
3. Preview normalised WBS, activities, dates and dependency count.
4. Correct row-level validation errors.
5. Commit once all blocking checks pass.
6. Baseline Gantt renders **from database data**, not hardcoded JSON.

A worker reports while offline

1. Open the installed PWA from cache.
2. Type "Zone 3 pump P201 alignment done today."
3. Save; the device assigns `client_event_id` and writes to IndexedDB.
4. UI shows "Saved locally — waiting for connection."
5. Connection returns; the queue syncs automatically or via Retry.
6. The server acknowledges the same ID; the device marks it synced.
7. AI creates a proposal; the reporter sees "Waiting for review."

A planner uploads a DPR

1. Upload the daily XLSX.
2. Select the relevant sheet; map description / date / zone / discipline columns.
3. Preview row segmentation.
4. Submit; each row becomes an attributable field update.
5. Processing summary shows proposed, clarification, unmatched and failed counts.

A site engineer reviews

1. Open the highest-priority proposal.
2. Compare raw message, photo, extracted event, top candidate, alternatives and forecast effect.
3. Confirm, correct or reject.
4. The transaction writes the approved fact, recalculates the schedule and appends audit.
5. UI reports success and links to the changed activity.

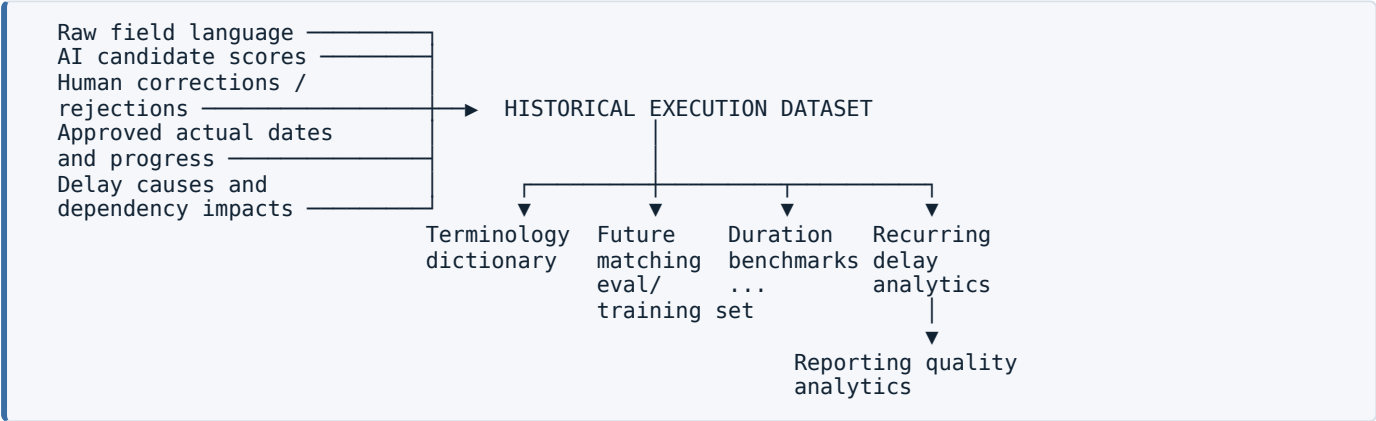
A manager investigates a delay

1. Dashboard shows projected finish +4 days.
2. Click the delay chain.

- 3. Inspect trigger actual, affected successors, old/new dates and reason.
- 4. Open activity history to see evidence and the reviewer's decision.

7.2 Institutional Memory

What is retained



Questions this layer can eventually answer

- Which field phrases frequently map to each WBS activity type?
- Which contractors or disciplines create the most ambiguous updates?
- Where does approved actual duration consistently exceed planned duration?
- Which delay causes recur by zone, crew, discipline or activity type?
- How often does the reviewer correct the top AI match?
- **Does confidence actually predict correctness?**
- Which granular milestones best predict final completion?
- Which predecessor delays create the largest downstream effect?

MVP analytics cards

Storage is mandatory; the analytics screens are Tier 2.

Match quality		Schedule health	
Top-1 accuracy	84%	Planned completion	68%
Top-3 accuracy	96%	Actual completion	61%
Correction rate	11%	Variance	-7 pp
Low-conf. abstain	9%	Project finish	+4 d
Data quality		Review operations	
Unmatched updates	3	Pending proposals	12
Missing asset tags	14%	Median review time	18 s
Duplicate retries	0	Rejected proposals	4

All displayed numbers must come from the current database or a **clearly labelled** illustrative fixture.

7.3 Quality Gates

Area	Metric	Target	Failure meaning
Activity matching	Top-1 accuracy on held-out set	≥80%	matcher claim too weak; constrain or reduce schedule
Candidate usefulness	Top-3 accuracy	≥95%	correction UI may not save reviewer time
Unsafe certainty	False-confident rate on wrong matches	<5%	confidence calibration unsafe
Event extraction	Event-type exact match	≥90%	START/FINISH semantics unreliable
Asset/zone extraction	Exact / normalised accuracy	≥90%	candidate filter loses signal
Abstention	Ambiguous cases marked clarification	≥80%	model guesses too often
Granularity	Gold parent + subtask match	≥85%	central differentiation not proven
Human gate	Unapproved writes to actuals	0	release blocker
Idempotency	Duplicate decision/update applications	0	release blocker
Scheduling	Hand-calculated scenarios passed	100%	release blocker
Import	Valid fixture rows imported	100%	data foundation broken
Import safety	Invalid/cyclic fixture committed	0	release blocker
Offline	Captured updates lost across reload/reconnect	0	offline claim invalid
Traceability	Approved actuals with source + reviewer + time	100%	audit claim invalid
Run repeatability	Consecutive reset-to-end runs	2	not walkthrough ready

7.4 Functional Test Matrix

ID	Scenario	Expected result
F01	clean FINISH text	correct proposal + actual finish after confirm
F02	"about 60% done"	progress 60 proposal
F03	two events in one message	two reviewable events
F04	ambiguous "piping complete"	clarification / alternatives, no write
F05	activity ID invented by model	validator rejects
F06	correction to second candidate	corrected activity applied; original retained
F07	reject	zero change to actuals / projected dates
F08	confirm twice	one actual and one schedule effect
F09	already-complete target	warning or clarification
F10	offline save + reload	update remains queued
F11	timeout then retry	one server update
F12	invalid import predecessor	row-level error; no commit
F13	cycle in dependency graph	import / recalculation rejected
F14	granular milestone repeated	no double-earned progress
F15	parent final milestone complete	FINISH proposed only when required milestones satisfied
F16	3-day predecessor delay	exact downstream shift matches hand calculation
F17	photo-only input	description / evidence shown; no autonomous completion claim
F18	malformed AI response	safe error / clarification; app remains usable
F19	batch with one bad row	clear partial-processing policy and traceable error
F20	demo reset	returns all fixture tables to known state

Non-functional checks

- Field capture touch targets and text are usable on a small phone.
- App shell opens in airplane mode after the first online load.
- Dashboard remains readable at projector resolution.
- 40-task Gantt loads and stays responsive.
- **API keys are absent from the frontend bundle and network-visible client configuration.**
- Errors identify what the user can do next.
- The review queue never shows only a spinner with no timeout or fallback.
- Accessibility: colour is supplemented by words or icons; contrast is adequate.

7.5 Definition of MVP Completion

The MVP is complete only when **every** statement below is true.

Functional completeness

- [] A user imports a valid schedule CSV/XLSX through the app
- [] The schedule is stored as WBS nodes, activities and dependency edges
- [] A worker can submit text while online or offline
- [] DPR/Excel rows enter the same canonical pipeline
- [] The system extracts actual START/FINISH/progress/delay meaning
- [] The system ranks valid L5/L6 candidates and never accepts invented IDs
- [] Multiple small field events can map to one larger planned activity
- [] Confidence, alternatives, evidence and clarification are visible
- [] Confirm, Correct and Reject all work
- [] No unapproved proposal changes an actual or forecast
- [] Approved actuals recalculate downstream FS dependencies
- [] Dashboard and Gantt visibly update Plan vs Actual
- [] Every decision and downstream change is auditable
- [] Approved and rejected data remains queryable as historical knowledge

Reliability completeness

- [] Offline updates survive reload and synchronise without duplication
- [] Confirm is transactional and idempotent
- [] The dependency engine passes every hand-calculated test
- [] Bad imports, malformed AI outputs and network failures have safe states
- [] Demo reset works repeatedly

Evidence completeness

- [] Matching and extraction metrics measured on a frozen set
- [] Synthetic data labelled honestly
- [] Simulated chat/SMS and unavailable integrations labelled honestly
- [] Two clean end-to-end runs completed after reset
- [] The walkthrough delivered once by Soumyak and Arin
- [] Retrospective written

Passing some screens is not enough. **The acceptance unit is the end-to-end data journey.**

7.6 Success Criteria

User success

- A field reporter can capture an update in **under 30 seconds**.
- No update is lost because data service disappears temporarily.
- A reviewer can confirm a clear proposal in **under five seconds**.
- A planner can trace every actual back to its source and human decision.

- A manager can see current variance and its cause without opening raw DPRs.

Technical success

- Top-1 matcher accuracy $\geq 80\%$ on the frozen 60+ message set.
- Wrong high-confidence matches stay below the defined threshold.
- No LLM-generated ID bypasses validation.
- **Zero** unapproved or duplicate actual writes.
- All critical schedule-cascade tests match independent calculations.
- Offline capture loses **zero** tested updates.

SIH demo success

- Someone outside the team understands the problem and differentiator within **60 seconds**.
- The working system appears within the first **90 seconds**.
- The cascade moment clearly shows cause and effect.
- The team can distinguish real, synthetic, simulated and future components.
- Any team member can explain the one-sentence product description consistently.
- Every member has run the full end-to-end flow themselves at least once.

Target scorecard

Core pipeline completeness	[████████████████████]	100%
Human-gate enforcement	[████████████████████]	100%
Schedule test pass rate	[████████████████████]	100%
Matcher top-1	[███████████████████]	80%+
Matcher top-3	[███████████████████]	95% target
Offline retention	[████████████████████]	100%
Run repeatability	[████████████████████]	2/2 runs

7.7 Q&A Defence — Everyone Learns These

"How do you know the AI matched correctly?"

We constrain candidates, return alternatives, measure top-1 and top-3 on a held-out set, calibrate confidence, validate all IDs server-side, and require an authorised human decision.

"Why not just upload the DPR to a chatbot?"

A chatbot gives you none of: a persistent schedule graph, constrained candidate retrieval, granular parent/subtask state, deterministic dependency propagation, transactional approval, evidence lineage, idempotency, or an institutional dataset of corrections.

"Is scheduling done by AI?"

No. AI interprets language and proposes links. Scheduling uses deterministic graph and date logic that is independently tested.

"Does a photo prove completion?"

No. It is human-reviewable evidence. The optional AI caption describes visible facts and never certifies completion.

"Is this real Oil India data?"

No. The MVP uses a synthetic but structured EPC schedule because production data was not supplied.

The data model and import layer are designed to accept validated schedule exports.

"Does it integrate with Primavera?"

The MVP imports CSV/XLSX schedule exports. Direct API integration and writeback are roadmap items. Approved actuals can also be exported in a controlled format.

"Does it work offline?"

Field capture and the outbox work offline. AI processing, multi-user review and shared schedule synchronisation run when connectivity returns.

"Does it work on button phones?"

The architecture supports low-bandwidth transports — SMS, IVR, USSD — feeding the same adapter contract. The five-day build may simulate SMS; real telecom integration is not claimed.

"What breaks at scale?"

Candidate retrieval must become more sophisticated as activities grow; review queues need calibrated automation and triage; LLM cost and latency must be controlled; production deployment needs enterprise security, calendars, permissions and integration testing.

"How did you build this without writing code by hand?"

Every line is AI-generated and human-reviewed. The application is generated in Lovable against a schema we froze first; the backend and Edge Functions are generated in Claude Code and reviewed line by line by our technical lead. Humans direct, read, verify and test. Wrong output is corrected by re-scoping the generation request smaller, not by hand-patching.

"Why is this useful beyond one project?"

The adapter-based architecture and project-scoped schema generalise to EPC and infrastructure organisations. Approved mappings and durations create organisation-specific knowledge over time.

7.8 Worked Demo Data

Schedule excerpt

Activity ID	WBS	Activity	Zone	Planned	Pred.	Status
CIV-L6-103	PRJ.CIV.Z3.FDN	Foundation concreting	Zone 3	18-20 Aug	CIV-L6-102	in progress
MEC-L5-201	PRJ.MEC.Z3.P201	Install Pump P-201	Zone 3	21-25 Aug	CIV-L6-103	not started
PIP-L6-301	PRJ.PIP.Z3.P201	Connect suction/discharge piping	Zone 3	26-29 Aug	MEC-L5-201	not started
ELE-L6-401	PRJ.ELE.Z3.P201	Terminate Pump P-201 feeder	Zone 3	26-28 Aug	MEC-L5-201	not started
COM-L6-501	PRJ.COM.Z3.P201	Pump P-201 test run	Zone 3	30 Aug	PIP-L6-301;ELE-L6-401	not started

One update, end to end

RAW:

"P201 alignment completed today. Electrical termination pending due to gland shortage."

EXTRACTED EVENT A:

FINISH • Pump P201 alignment • 22 Aug • Mechanical • Zone 3
→ Parent MEC-L5-201 / milestone ALIGN • High 94%

EXTRACTED EVENT B:

BLOCKED • Pump P201 electrical termination • 22 Aug
• Electrical • Zone 3
→ ELE-L6-401 • Medium 78% • reason: material shortage

HUMAN ACTIONS:

Confirm A
Correct/Confirm B after inspecting alternatives

SYSTEM EFFECTS:

Parent earned progress increases
Electrical task becomes blocked
Projected finish moves
Audit and history retain both decisions

7.9 Submission Artifact Checklist

- [] deployed MVP URL
- [] repository / source export and environment setup notes
- [] database schema / migrations
- [] synthetic schedule CSV and XLSX
- [] DPR and free-text fixtures
- [] 60-80 case evaluation dataset and frozen scorecard
- [] prompt / version manifest
- [] five independent scheduling test calculations
- [] automated / manual test results
- [] architecture diagram
- [] evidence and confirmation policy
- [] limitations / roadmap statement
- [] demo script and Q&A sheet
- [] reset instructions
- [] key screenshots and offline deck
- [] the Day 5 retrospective page

The Frozen Product Statement

We are building an offline-capable, AI-assisted planning-to-execution bridge that converts messy field updates and reports into structured START, FINISH, progress and delay events; matches them to the correct L5/L6 schedule activities; reconciles multiple small field tasks with larger planned WBS work; requires human confirmation; and then deterministically updates Plan vs Actual forecasts, downstream dependencies, audit history and institutional knowledge.

The single sentence every member must remember

AI sits between field execution and project control to propose trustworthy schedule-linked actuals; humans approve them, and deterministic code calculates their impact.

The final five-day rule

If the application cannot demonstrate:

FIELD INPUT

- STRUCTURED EVENT
- L5/L6 + GRANULARITY MATCH
- CONFIDENCE / ALTERNATIVES
- HUMAN DECISION
- VERIFIED ACTUAL
- DEPENDENCY CASCADE
- DASHBOARD + AUDIT + HISTORY

then the team has not built the MVP yet.

SIH 2026 · PS 26122 · Oil India Limited · Team build guide, per-person edition.

Day 1 Sunday 23 August 2026 — Day 5 Thursday 27 August 2026.